

Planificarea proiectului

Etapele proiectului

Etapa	Perioadă bonus	Perioadă punctaj normal	De când încep penalizările săptămânale
Etapa 0 (0.3)		02.03. 2026	
Taskuri etapa 0 (0.35) (0.025) Alegerea temei (titlul site-ului) - Descrierea succintă a temei (google docs, 1/2-1 pagina). Va contine următoarele informații: (0.025) Impartirea informatiilor, serviciilor etc. pe categorii si subcategorii. (0.025) Identificarea efectiva a paginilor (încercați să fie doar 4-5) și partial a legaturilor dintre ele (ce informatii vor avea pagini separate si care informații vor fi subsecțiuni în aceeași pagină) (0.025) Stabilirea cuvintelor/sintagmelor cheie (o lista cu cuvintele cheie ale site-ului cât și cate o lista pentru fiecare pagină în parte) (0.2-0.25) Căutarea unor site-uri similare (4-5 site-uri) ca tema pentru a observa modul de organizare a informației. Veti observa pentru fiecare site cum au impartit informatiile si cum au facut designul. Veti nota pentru fiecare site ideile demne de implementat, dar și neajunsurile acelor site-uri (pentru fiecare site minim 2 lucruri pro si 2 contra). Veți pune linkuri către ele în fișier.			
Etapa 1 (punctaj recomandat 1.1)	09.03. 2026	16.03. 2026	
Taskuri etapa 1 Creați prima pagină a site-ului (doar prima pagină; fără stilizare încă, fiindcă veți primi taskuri legate de acest aspect). Puteți pune în această pagină text care va fi mutat în alte pagini, mai târziu, dar nu faceți încă mai multe pagini fiindcă le vom genera prin Node! La prezentare vă rog să aveți pentru fiecare task notată linia din program la care l-ați rezolvat ca să nu dureze prezentarea mai mult de 3-4 minute. Tagurile și atributele cerute mai jos trebuie să fie relevante în raport cu conținutul și tema de proiect pentru a fi punctate. Punctaj total pentru cerințe: (0.025) Creați un folder al proiectului care va cuprinde toate fisierele necesare site-ului vostru. Creați în el un fisier numit index.html. Deschideți acest fișier cu un editor de text care marchează sintaxa. Adăugați în fișier doctype și setați limba documentului în tagul html (0.05) Adaugati un title corespunzător conținutului textului. Folosiți 4 taguri meta relevante pentru a specifica: charset-ul, autorul, cuvintele cheie, descrierea. Punctajul se va da în funcție de cât de precise și relevante sunt conținuturile tagurilor meta. (0.025) Textul din pagină trebuie să conțină toate cuvintele cheie decise pentru pagina curentă (și enumerate în tagul meta). Puteți găsi mai multe sintagme cheie pe care le puteți folosi, cu https://www.wordtracker.com/ sau https://app.neilpatel.com/en/ubersuggest/keyword_ideas. Acestea trebuie să apară de mai multe ori în pagină, în taguri relevante. (0.025) Creați un folder (de exemplu numit "resurse") care va conține toate fisierele folosite de site, dar care nu sunt pagini html (de exemplu imagini, fisiere de stilizare etc). In el creati un folder numit ico. Adaugati un favicon relevant pentru temă. Folosiți https://realfavicongenerator.net pentru a genera toate dimensiunile necesare de favicon și codul compatibil pentru diversele browsere și sisteme de operare. Pentru favicon transparent, trebuie sa setati si o culoare a tile-ului (de background), care trebuie specificata și în tagul meta: <meta name="msapplication-TileColor" content="...culoarea aleasa de voi..."> (0.025) Împărțiți body-ul în header, main, footer. (0.05) Folosiți minim un tag dintre: section, article, aside. Trebuie să existe măcar un caz de taguri de secționare imbricate (secțiune în secțiune). Puneți headingul cu nivelul corespunzător nivelului imbricării. Atenție, nu folosim headinguri decât ca titluri pentru tagurile de secționare. Observație: nivelul headingului trebuie să corespundă nivelului de imbricare a secțiunii (de exemplu un tag de secționare aflat direct în body are titlul scris cu h2, dar un tag de sectionare aflat într-un tag de secționare care la rândul lui se află în body, va avea titlul scris cu h3)			

7. (0.025) Minim un heading din pagină va fi acompaniat de un subtitlu, folosind tag-ul <hgroup>.
8. (0.05) În **header** faceți un sistem de navigare ca în curs (nav cu listă neordonată de linkuri), cu opțiuni principale (care vor reprezenta paginile site-ului) și secundare (pentru opțiunea "Acasă", adică pagina principală, subopțiunile vor cuprinde linkuri către secțiunile paginii, care vor avea id-uri relevante). Subopțiunile vor fi puse într-o listă **imbricată** în lista de opțiuni principale. Folosiți în header tagul h1 pentru titlul site-ului.
9. (0.025) În cadrul secțiunilor folosiți minim 2 taguri dintre următoarele taguri de grupare: p, blockquote, dl
10. (0.05) Veți crea o secțiune de evenimente, cu date și ore (folosind tagul <time> și atributul datetime cu informații de dată și oră). Evenimentele vor fi enumerate într-o listă ordonată () sau neordonată (). Evenimentele vor avea un nume pus în tagul , urmat de o descriere. De exemplu: 5 ianuarie ora 12, **Prajituri cadou** - oferim tuturor vizitatorilor câte o prăjitură gratis în funcție de stocul disponibil.
11. (0.05) Adăugați în pagină o imagine cu descriere, folosind figure și figcaption. Imaginea va avea și o descriere mai scurtă în atributul title (care se vede când venim cu cursorul pe ea). Pe ecran mic (mobil) trebuie să se încarce o variantă mai redusă în dimensiune (bytes) a imaginii, pe tabletă o variantă medie, iar pe ecran mare varianta cea mai mare a imaginii (folosiți tagul picture). Folosiți un editor grafic pentru cropping și redimensionare pentru a obține cele 3 variante de imagini.
12. (0.075) În cadrul textului îndepliniți **3** dintre cerințele de mai jos, **la alegere**:
 - a. marcați minim 3 cuvintele și sintagme cheie cu ajutorul tagului b
 - b. Marcați minim 2 cazuri de text idiomatic (termeni științifici, în altă limbă, termeni tehnici, de jargon, etc) cu tagul i. Trebuie să fie termeni relevanți pentru site
 - c. Realizați un text care anunță ceva urgent, important și marcați-l cu tagul strong
 - d. În cadrul textului folosiți un cuvânt în mod ironic, sau considerați că puneți accentul (la citire) pe un cuvânt din text. Marcați cuvântul accentuat cu em
 - e. marcați textul șters (corectat sau care nu mai e relevant) cu tagul s și textul inserat în loc cu tagul ins
 - f. marcați o abreviere (relevantă pentru site) cu tagul abbr și cu atributul title specificați sintagma abreviată
 - g. Alegeți un termen relevant temei, dar mai puțin cunoscut, dintre cei care apar pe site. Marcați acel termen definit cu dfn, și scrieți definiția pentru el
 - h. marcați un citat relevant pentru conținutul site-ului cu tagul q
 - i. marcați titlul unei lucrări citate cu tagul cite
13. (0.1=5*0.02) Creați următoarele linkuri (folosind tagul <a>), cu mențiunea că trebuie să fie relevante în pagină:
 - a. un link către o resursă externă (de exemplu un alt site cu temă relevantă pentru conținut). Acest link nu se va plasa în meniul. Linkul se va deschide într-un tab nou.
 - b. un alt link către un element dintr-o resursă externă (referențiat prin id). Exemplu: <http://ceva.com#referinta>. Textul tagului <a> va fi chiar adresa indicată de link. Adresa trebuie să fie mai lungă. Dacă linkul nu are loc pe rândul său, trebuie să se separe în bucăți trecând pe rând nou (vedeți exemplul din curs de la <wbr>)
 - c. un link în footer care duce către începutul paginii.
 - d. un link care conține o imagine. La click pe imagine, se va deschide aceasta în tabul curent, într-o versiune mai mare.
 - e. un link de tip download, care va seta numele fișierului de descărcat la altul decât cel inițial (de exemplu, se numea poza.png dar se descarcă automat cu numele poza_descarcata.png). Pentru ca descărcarea să funcționeze, pagina html trebuie să funcționeze pe un server. Descrieți extensia Live Server (pentru VSCode) și apăsați pe "Go Live" care apare în dreapta-jos (poate necesita restart după instalare).
14. (0.05) Creați un iframe în pagină care conține un videoclip de youtube embedded (vedeți: https://developers.google.com/youtube/player_parameters). Imediat deasupra iframe-ului într-un tag div veți crea minim două linkuri (tagul <a>) către alte videoclipuri de youtube și un al treilea link către videoclipul de youtube deja încărcat în iframe. Toate aceste linkuri se vor deschide în iframe (vedeți exemplul de la curs).
15. (0.1) Creați un tabel cu sens în site (de exemplu: cu oferte, cu programul de lucru, cu detalii despre evenimentele importante care urmează, cu date despre cei mai buni angajați, cu date despre niste proiecte în care a fost implicată firma pentru care faceți site-ul, cu date despre cele mai vândute produse etc.). Tabelul trebuie să fie împărțit în thead (antet), tbody, tfoot (fiecare conținând date relevante). În antet vor fi celule cu numele coloanelor (folosiți th). Tabelul trebuie să aibă minim 5 rânduri și 4 coloane. Minim una dintre celule va avea rowspan sau colspan setat la valoare diferită de 1. Tabelul va avea și descriere (caption).
16. (0.025) Creați în pagină mai multe zone de details și summary. Pot fi întrebări frecvente, pot fi niște oferte pentru care afișăm titlul și utilizatorul le deschide pe cele care îl interesează, pot fi secțiuni explicative etc.
17. (0.05) Folosiți în site tagul meter (de cel puțin două ori în pagină) setând o valoare existentă, minimul, maximul, de la ce punct se consideră valoare mică, de la ce punct se consideră valoare mare și care e valoarea optimă. Un tag meter va avea o valoare mică (până în limita low) iar celălalt valoare mare (peste valoarea atributului high). Exemple de folosire: număr de participanți confirmați la evenimente, review-uri date de ziare, utilizatori etc cu o notă din partea lor, numărul de clienți într-un anumit interval orar în raport cu capacitatea magazinului etc.)
18. (0.1 = 4 * 0.025) În footer se vor adăuga cu ajutorul tagului address informații de contact:
 - a. telefon fictiv, marcat cu tagul <a> și URI Scheme-ul corespunzător
 - b. adresă fictivă care la click deschide o locație pe Google Maps (locația în mod normal ar corespunde cu adresa dar voi veți pune drept locație în maps, Facultatea de Matematica și Informatică). Adresa fictivă va fi separată cu
 pe două sau mai multe rânduri.
 - c. e-mail fictiv, marcat cu tagul <a> și URI Scheme-ul corespunzător în href

- d. Link care deschide o aplicatie de comunicare precum [whatsapp](#) pentru chat
19. (0.05) În footer se va adăuga informație de copyright, folosind tagul small, simbolul specific de copyright cu codul html necesar (forma &cod;) și data creării paginii scrisă în limba română și pusă în tagul time cu atributul **datetime** corespunzător (doar cu informații despre dată, nu și oră).
20. (0.15) **Cerință obligatorie!** Pagina trebuie sa fie validă din punct de vedere sintactic. Deci verificați cu [validatorul html](#). Validatorul va fi pregătit într-un tab, la prezentare, și pagina se va valida pe loc. **Doar după ce s-a demonstrat că pagina e validă se începe prezentarea.**

Se poate da bonus pentru o temă bine făcută sau pentru folosirea mai multor taguri decât minimul specificat (dar studentul trebuie să anunțe la prezentare că le-a folosit)

Evitați pentru moment să adăugați alte taguri fiindcă vor apărea în taskurile următoare.

Bonusuri:

- Bonus (0.05-0.2 in functie de complexitate)** Folosirea unei formule scrise în MathML - formula trebuie să aibă sens în contextul site-ului.
- Bonus (0.05)** Afișarea unui pdf în pagină cu ajutorul tagului <embed> sau <object>
- Bonus (0.2-0.3 in functie de complexitate)** Crearea unei hărți de imagini, folosind tagurile <map>, <area>
- Bonus (0.05)** Adăugați pentru adresa facultății (de la un subpunct anterior) și un iframe cu locația marcată pe google maps.
- Bonus (0.05)** Crearea unui iframe în care se afișează pe rând (în mod automat) videoclipurile dintr-un playlist de pe youtube (playlist-ul va avea minim 2 videoclipuri), cu butoanele de controlare a videoclipurilor afișate. Playlist-ul se va relua automat de la început după redarea ultimului videoclip. Cerința se va rezolva doar din parametri în cadrul linkului youtube. Documentație: https://developers.google.com/youtube/player_parameters

Etapă 2 (punctaj recomandat 0.7)

18.03. 2026

23.03.2026

Taskuri etapă 2 (punctaj recomandat:-0.4)

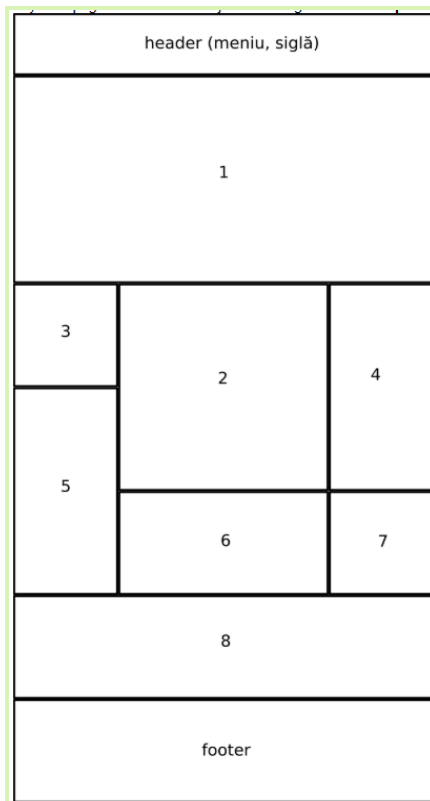
Dacă stilizarea dintr-o cerință nu va place, puteți să imi cereți o altă variantă (imi scrieți pe chat). Culoarele din imaginile și videoclipurile date ca exemplu nu trebuie respectate (folosiți culoarele din schema cromatică aleasă de voi).

(0.05) Se va face un repository local și se va pune proiectul pe Github.

(0.05) Task schema cromatică: Alegeți o culoare de bază reprezentativă pentru site-ul vostru. Alegeți o schemă cromatică de minim 5 culori folosind site-ul: <https://color.adobe.com/create/color-wheel> având drept culoare principală(culoarea din mijloc, în cadrul toolului) culoarea aleasă de voi. Specificați în fișierul-documentație (creat în etapa 1) schema cromatică aleasă și motivul alegerii aceluia tip de culoare (referindu-vă la tema site-ului și psihologia culorilor) În interiorul tagului body, pentru cele 5 culori alese definiți variabile pentru culori de background, de text, de border, de highlight etc. Veți folosi în CSS **doar** valorile variabilelor, nu valorile specificate propriu-zis. Puteți alege și mai mult de 5 culori dar să fie toate în armonie cu culoarea principală a schemei cromatice.

(0.15) Task layout: Se va realiza un layout responsive pentru prima pagină (pentru celelalte pagini alegeți voi layoutul). Pentru layout aveți următoarele cerințe:

- Layoutul paginii se va face cu ajutorul unui grid **care va cuprinde conținutul din mainul paginii** (nu e obligatoriu ca headerul și footerul să fie și ele într-un grid). Pe ecranul mare layoutul va avea forma:



- Atenție imaginea indică doar harta gridului (rândurile și coloanele) nu și dimensiunile. Nu trebuie să respectați faptul că o coloană din grid pare să fie de aceeași lățime cu altă coloană din grid; idem pentru rânduri.
 Veți stabili voi forma pentru ecranul mediu și cel mic, cu condiția să schimbați forma pe ecranul mediu (să nu se afișeze la fel ca pe ecranul mare, iar pe ecranul mic trebuie să nu mai existe coloane).
 Rolul propus al zonelor (nu e obligatoriu, puteți decide alt rol pentru ele) din imagine este următorul (pentru zonele unde se specifică să puneți un placeholder, veți folosi un text de tipul "[Lorem ipsum](#)":
 - Zona 1 este pentru prezentare. Este primul lucru pe care îl vede utilizatorul când intră pe pagină. Va avea un rol de poster cu text sau imagini atractive care să descrie scopul site-ului. Alegeți unul dintre tipurile de layout în funcție de conținut:
 - Z-pattern** pentru un conținut cuprinzând mai mult imagini
 - F-pattern** dacă zona 1 cuprinde în special text.
 - Zona 2 este pentru tabel.
 - Zona 3 este pentru calendarul de evenimente. Veți pune un placeholder.
 - Zona 4 este pentru anunțuri. Veți pune un placeholder.
 - Zona 5 este pentru lista de utilizatori online. Placeholder.
 - Zona 6 este pentru date despre utilizator și statistici. Placeholder.
 - Zona 7 este pentru date despre site și server. Placeholder.
 - Zona 8 este pentru diverse alte tipuri de conținut (ce considerați că ar mai trebui să aibă pagina; textul scris în etapa 1 etc.).**Pe aceasta o puteți stiliza așa cum vă place vouă și veți primi punctaj pe aspect și creativitate.**
- Atenție, o pondere mare din punctaj este dată de faptul că layoutul nu apare tăiat (de exemplu din cauza unui overflow: hidden pe un container cu conținut prea mare și nu există scrollbar orizontal pe viewport (se acceptă scrollbar orizontal doar unde e cerut în mod explicit de către un task)).** Gridul va fi definit prin grid-template-areas.
- Scrisul trebuie să descrească odată cu redimensionarea paginii (pe ecran mic să fie scris mic, pe ecran mediu scrisul un pic mai mare, iar pe ecran mare scrisul să fie un pic mai mare decât pe mediu. Diferențele vor fi mici de 1-2px. **În afară de font-size-ul html-ului restul dimensiunilor de text se vor da în unități relative (em, rem, vw etc.).** (Des)creșterea dimensiunii textului de la o dimensiune la alta de ecran se va face printr-o tranziție.
- Animațiile (cu excepția celor cerute în mod special, în alte taskuri) vor fi dezactivate pe ecran mic (eventual și mediu, dacă doriți).

Important! La prezentare veți fi întrebați care sunt principiile de design și ierarhie vizuală folosite (și va trebui să justificați cum anume le-ați aplicat în pagină).

(0.25) Task design rudimentar:

- Folosind variabile css, să se adauge o spațiere în stânga și dreapta paginii, comprimând un pic conținutul, dar aerisind pagina. Spațierea trebuie să fie identică în cele două direcții. Spațierea va fi mai mică pe ecran mediu și de valoare minimă pe ecran mic.
- Folosiți gap pentru a realiza o spațiere între celulele gridului paginii. Spațierea trebuie să descrească pe ecran mediu și mic.
- Izolați vizual zonele paginii: header, footer, și zonele gridului folosind minim 3 dintre următoarele efecte css: background de culoare (aleasă din schema cromatică) diferită pe zone diferite, border, colțuri rotunjite ale box-ului, box-shadow
- Folosiți padding pentru a distanța textul din zone de granițele zonelor. Padding-ul ar trebui să fie egal pentru toate zonele de text. Folosiți variabile CSS în cazul în care nu puteți asigura acest lucru prin simplii selectori.
- Elementele media (imagini, videoclipuri etc.) care vin cu o lățime presetată vor primi lățimea în procente, setând, totuși, o lățime maximă și minimă pentru ele, pentru a nu avea efecte vizuale nedorite. Aceste valori pot să difere în funcție de dimensiunea ecranului

(0.05) Task iconuri și font extern. Folosiți în prima pagină a site-ului un [font extern prin Google API](#). Folosiți în pagină, într-un loc relevant un icon static și unul animat (**diferite de iconurile cerute în eventuale alte taskuri**) din colecția [Font Awesome](#) (fie folosind kit - recomandat - fie folosind linkul CDN ca în exemplu: <https://replit.com/@IrinaCiocan/curs4-example#fontawesome.html>)

(0.05) Stilizare tabel:

Tabelul trebuie să aibă **minim 4 rânduri și 4 coloane**.

Minim una dintre celule va avea rowspan sau colspan setat la valoare diferită de 1

Se va stiliza tabelul din fișierul HTML astfel:

- Tabelul va avea captiune (caption) poziționată sub tabel (folosind css).
- Borderele dintre doua coloane vor avea culori alternate (borderele care au în stânga o celulă cu număr par și în dreapta o celulă cu număr impar vor fi de culoarea c1, iar borderele care au în stânga o celulă cu număr impar și în dreapta o celulă cu număr par vor avea culoarea c2). Culorile le alegeți voi în funcție de tema site-ului.
- Grosimea borderului care desparte zona de thead de cea de tbody, respectiv zona de tbody de cea de tfoot va fi de 2 ori sau de 3 ori mai mare (alegeți voi cât de mare să fie în funcție de aspect) decât grosimea borderelor (minim 1px) dintre rânduri obișnuite.
- Când se vine cu cursorul pe un rând, apare treptat în interiorul lui un box-shadow mai grosuț (ca să se vadă bine), complet suprapus pe rând (adică nedeplasat în stânga sau dreapta). Tranziția apariției va dura jumătate de secundă.
- Pe ecran mic (eventual și mediu), tabelul va fi într-un container cu un scrollbar orizontal (atenție doar containerul cu tabelul va avea scrollbar, nu toată pagina). Hint: tabelul va fi pus într-un div cu un anume tip de overflow orizontal)

Indicații de rezolvare:

- Pentru taskul cu valori alternate folosiți pseudoclasele :nth-child(odd), :nth-child(even) cu elementele cerute (td pentru celule(coloane) și tr pentru rânduri)
- Proprietățile de care aveți nevoie sunt în special la [CSS Styling Tables \(w3schools.com\)](#)
- Bara de scroll se poate realiza cu ajutorul proprietății overflow.

(0.05) Task stilizare-taburi:

Linkurile către iframe se vor stiliza asemănător cu ce se vede în videoclipul de mai jos. Încerați să identificați din videoclip elementele de stil ce trebuie aplicate. Dreptunghiul alb cu border este iframe-ul (în care eu nu am mai pus vreo pagină). În rezolvare e obligatoriu să folosiți undeva și flexbox.



Indicații de rezolvare:

1. Creați un container în care puneți linkurile care se deschid în iframe (cel care are clasa "taburi-iframe" în exemplul de mai jos). De asemenea grupați containerul linkuri și iframe-ul într-un alt container (cel cu clasa container-iframe din exemplul de mai jos)

```
<div class="container-iframe">
  <div class="taburi-iframe">
    <a href="https://www.youtube.com/embed/qlhldlfojmc" target="ifr-video">Reteta tort vanilie</a>
    <a href="https://www.youtube.com/embed/dsJtgmAhFF4" target="ifr-video">Reteta tort ciocolata</a>
    <a href="https://www.youtube.com/embed/Wtxbt-CpA2s" target="ifr-video">Reteta tort căpșuni</a><br/>
  </div>
  <iframe name="ifr-video" width="560" height="315" src="https://www.youtube.com/embed/qlhldlfojmc"
    frameborder="0" allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture"
    allowfullscreen></iframe>
</div>
```

2. Stilizați tagurile <a> sub forma de butoane (width și height setat, background, border)

3. Aplicați display: flex pe containerul care trebuie să conțină coloane (fie container-iframe, fie taburi-iframe în funcție de cerință)

(0.05) Link top:

Linkul către începutul paginii trebuie să îndeplinească următoarele cerințe:

- Nu se vor folosi deloc imagini în realizarea acestui task!
- va avea forma de mai jos (obligatoriu, nu veți folosi culorile din videoclip, ci le veți seta conform schemei cromatice alese pentru site-ul vostru; nici nu e nevoie să aveți același dimensiuni - width și height - pentru elemente):



Observăm că atunci când venim cu cursorul pe el se petrece o tranziție. În videoclip tranziția e mai lentă pentru a se observa ușor detaliile, dar voi veți avea un timp scurt 300-700ms ca să nu devină neplăcut pentru utilizator.

Indicații de rezolvare: Linkul e de tip block sau conține un alt element de tip block pentru care bordura este rotunjită. Are un background în gradient, de la o culoare închisă la una mai deschisă și apoi iar aceeași culoare închisă. Borderul elementului este liniat (dashed). Săgețica e creată cu ajutorul unui element din link care conține caracterul ↑(cod: ↑). Săgețica este centrată orizontal și vertical. La venirea cu cursorul pe element, se rotește elementul rotund (ca în videoclip), săgețica rămânând nemișcată. Revine, apoi, prin rotire inversă, la starea inițială când luăm cursorul de pe el

- va avea o poziție fixă, în josul paginii, în stânga sau în dreapta
- Când venim cu cursorul pe el se va afișa un tooltip cu un text care explică că e butonul care duce în susul paginii (veți pune propriul vostru text în locul lui "text explicativ")
- Butonul e semi-transparent (proprietatea opacity), pentru a nu ascunde părți din pagină. Devine opac când venim cu cursorul pe el.

Indicație de rezolvare: Porniti de la exemplul: [exemplu_layout-1 - Replit](#) - cautati in html elementul cu id-ul "link-top", si fiul lui care e un div cu id-ul "triunghi". Preluati codul css pentru ele si il modificati pentru a obtine rezolvarea la acest task.

Bonusuri:

- (0.05 - 0.15 in functie de complexitate) Resetarea css-ului cu redefinirea spațiilor, dimensiunilor, culorilor, stilurilor bol si italic, eventual a bulleturilor și indicilor de listă, a stilului tabelor). In afară de body și html care vor primi dimensiuni în unități fixe, toate celelalte elemente vor folosi unități relative. Se vor folosi variabile pentru valori care se repetă și depind logic unele de altele. Puteti folosi de exemplu: <https://meyerweb.com/eric/tools/css/reset/> Resetarea css-ului se va face intr-un fisier separat
- (0.05-0.2 in functie de complexitate) Stilizarea unei formule scrise în MathML: părți diferite din formulă trebuie să aibă stil diferit, de exemplu schimbarea culorii, a dimensiunii de font, italicizarea sau îngroșarea caracterelor etc.)

Etapa 3 (punctaj recomandat 0.4)

30.03.2026

02.04.2026

Taskuri etapa 3

(0.25) Task meniu:

Se va implementa un meniu derulant al site-ului care va fi prezent pe fiecare pagină și va lega toate paginile între ele. **Nu e obligatoriu să folosiți aceleași culori, fonturi, dimensiuni ca în enunț și imagini/videoclipuri.**

Cerinte HTML. Meniul va fi implementat cu ajutorul tagului <nav> care contine o lista neordonata cu linkuri.

Pentru fiecare opțiune principală a meniului veți adăuga un icon reprezentativ din Font Awesome.

Meniul va prezenta minim două submeniuuri dintre care:

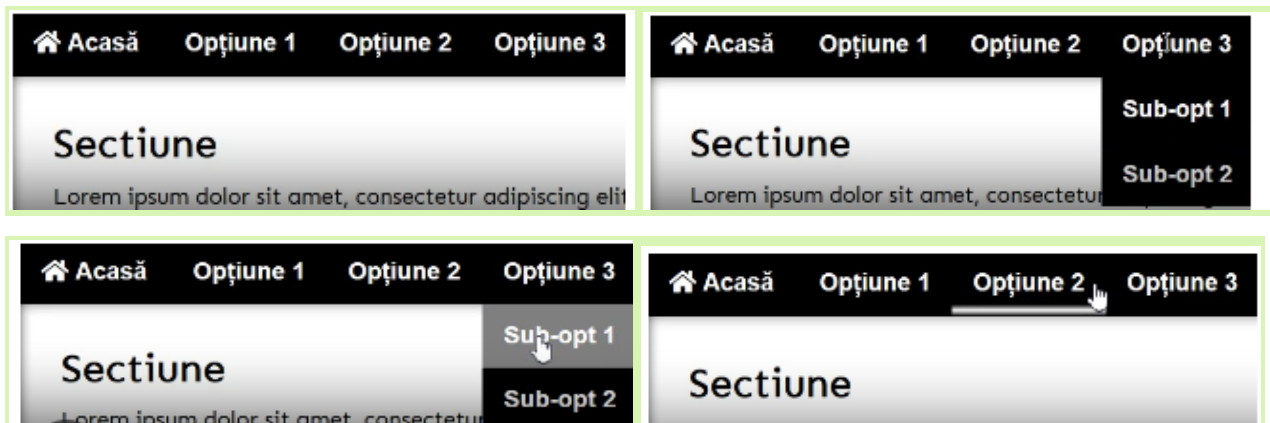
- unul conține linkuri către **alte pagini din site** (chiar dacă nu există încă). Aceste pagini sunt grupate în meniu sub un nume descriptiv (exemplu: opțiunea *galerii* cu subopțiunile *galerie statică* și *galerie dinamică*).
- Al doilea conține **linkuri către secțiuni dintr-una din pagini**. Nume opțiunii care deschide submeniul va fi numele acelei pagini.

Cerinte SASS/CSS. Stilul pentru meniu se va implementa în SASS și va fi compilat în CSS având în vedere următoarele cerințe:

- Pentru realizarea selectorilor complecși, se va folosi **imbricarea din SASS**
- Pentru blocuri de cod similare, în care diferă doar valorile proprietăților, se va folosi **@extend și, dacă e nevoie, instrucțiuni repetitive: @for, @while.**
- Se vor folosi variabile SASS sau CSS pentru orice valoare repetitivă.
- Puneți în variabile CSS (declarată în selectorul body) orice informații care ar putea depinde de tema cromatică a site-ului (pentru a o putea schimba ușor).

Se va realiza meniul pentru **ecran mare**, cu următoarele cerințe:

- Când venim cu cursorul pe o opțiune din meniu, se va realiza tranziția indicată în videoclip: cu ajutorul unui element ::before sau ::after se va genera o bara cu un background în gradient cu 3 culori (în exemplu gradientul este: negru, alb, negru). Când venim cu cursorul pe opțiune, bara coboară devenind din total transparentă, opacă, și se aliniază cu marginea de jos a butonului.
- Deschiderea submeniului se va face prin următoarea tranziție: se va scala de la 0 la înălțimea și lățimea totală (deci scalarea va fi pe verticală și orizontală) pornind din centrul boxului corespunzător submeniului.
- Când venim cu cursorul pe o opțiune din submeniu culoarea de background a opțiunii se va schimba.



La redimensionarea paginii, pe **ecran mediu**, meniul își va schimba afișarea. Textul se va face mai mic, iar pentru opțiunile principale din meniul vor rămâne doar icon-urile din FontAwesome (textul va fi ascuns)

Pentru **ecran mic**, se vor ascunde opțiunile afișându-se într-un hamburger menu. Vedeți exemplul de layout. În afișarea *burger* submeniul se va deschide ca mai jos, înlocuind opțiunea pentru prima pagină cu un icon-căsuță. La apariția submeniului, acesta va deveni vizibil cu ajutorul unui clip-path circular, care crește din colțul stânga-sus al submeniului până se afișează meniul complet. Tranzițiile pe opțiunile meniului se schimbă, dispărând bara glisantă, unicul comportament pe orice opțiune din meniul și submeniul fiind schimbarea culorii de background.



(0.15) Stil printare:

Observație: banner-ul care e menționat în mai multe exerciții/subpuncte este un paragraf (aflat în footer) cu textul "Acesta este un proiect școlar!".

Stilul pentru printare va avea următoarele lucruri implementate:

1. Bannerul să fie afișat în susul primei pagini, centrat pe orizontală, întins pe 80% din lățimea paginii, într-un border negru punctat
2. imaginile, videoclipurile, iframe-urile și alte elemente media împreună cu elemente dependente de ele (de exemplu descrierile imaginilor) vor fi ascunse. Linkul către începutul paginii va fi de asemenea ascuns
3. Gridul paginii va fi afișat ca *block* ca să nu apară celulele secționate în paginile printate.
4. toate linkurile din pagină se vor afișa ca text normal (nu colorate sau subliniate)
5. Headingul de nivel 1, cu titlu site-ului, va fi pe prima pagină, centrat pe verticală și orizontală. Va fi subliniat și cu litere de 2.5 ori mai mari decât dimensiunea de font a html-ului. Meniul va fi ascuns. După heading va fi un page break. Înaintea footerului, de asemenea va fi un page break

6. Pe **fiecare** pagină va exista un "watermark" care va conține numele vostru. Acesta va avea lățime de 100px și înălțime de 2cm, border dublu, și opacitate 0.65
7. La printare, paginile din stânga (în print preview) să aibă margine în stânga de 2.5cm și margine în dreapta de 0.5cm, iar cele din dreapta să aibă margine în dreapta de 2.5cm și 0.5cm în stânga.

Bonusuri:

- (0.05) Icon-ul meniului "hamburger" să fie creat cu 3 divuri/span (eventual puse într-un div container în locul unei imagini), cele 3 divuri vor primi background, width și height pentru a simula dreptunghiuri și vor fi poziționate absolut în interiorul containerului.
- (0.05) Când se trece pe ecran mic și apare iconul pentru meniu, apariția să fie făcută printr-o animație asupra divurilor care să implice schimbarea tuturor următoarelor proprietăți: culoarea celor 3 bare, o transformare geometrică, opacitate. Puteți schimba și alte proprietăți dacă doriți. Animația trebuie să aibă **minim 3 cadre cheie**.
- (0.05) Pentru bonusul anterior, fiecare bară din hamburger-menu să aibă asociată o animație, însă animațiile să înceapă succesiv cu o diferență de t milisecunde (de exemplu t=300). Delayurile diferite în cadrul animației se vor genera cu o instrucțiune for scrisă în sass.

Etapă 4

06.04.2026

20.04.2026

Taskuri etapa 4 (punctaj recomandat 1.15p)

(1p) **Trecerea site-ului pe node** și crearea de fișiere EJS conform cerințelor:

1. În folderul proiectului dați comanda "**npm init**" și setați numele, autorul, descrierea și cuvintele cheie pentru proiectul vostru. Instalați express și ejs.
2. Se va crea în rădăcina proiectului un fișier index.js. În el se va crea un **obiect server express** care va asculta pe portul 8080. (sau alt port dacă aveți deja folosit 8080)
3. Să se afișeze calea folderului în care se găsește fișierul index.js (**__dirname**), calea fișierului (**__filename**) și folderul curent de lucru (**process.cwd()**). Sunt **__dirname** și **process.cwd()** același lucru întotdeauna?
4. Se va folosi EJS pentru generarea (randarea) paginilor. Se va face un **folder numit views** în rădăcina proiectului. În el veți face un folder numit **pagini** (care conține paginile întregi) și altul numit **fragmente** (care conține părți de pagini (bucățele de cod html) ce pot fi refolosite pe mai multe pagini). Instalați în Visual Studio Code extensia **EJS language support**.
5. Din index (care va fi redenumit **index.ejs**) se vor decupa headerul și footerul și se vor pune în ejs-uri separate. De asemenea se va decupa partea de head care conține codul care nu se schimbă în funcție de pagină (de exemplu, tagul meta cu encodingul sau autorul, includerea faviconului, fișierelor css generale (nu specifice paginii) a scripturilor generale etc). Se va folosi funcția **include()** în fișierele ejs, pentru a include toate aceste fragmente în pagini
6. Se va realiza (dacă nu l-ați făcut deja) un **folder special cu toate resursele site-ului** (în stilul exemplului de la curs în care am pus toate fișierele statice, precum imagini, fișiere de stil, videoclipuri etc în folderul "resurse"). Numele folderului îl decideți voi, însă va trebui să fie structurat, de asemenea, în subfoldere în funcție de tipul și modul de utilizarea al fișierelor. Se va defini în program acest folder ca fiind **static**
7. Se vor schimba **căile fișierelor-resursă** folosite în pagini, astfel încât **să nu mai fie relative** ci stil cerere către server (de exemplu, /resurse/stiluri/ceva.css în loc de, de exemplu, ../resurse/stiluri/ceva.css)
8. Prima pagină (index) trebuie să se poată accesa atât cu **localhost:8080** cât și cu **localhost:8080/index**, **localhost:8080/home**. Realizați acest lucru folosind un vector în apelul app.get() care transmite pagina principală
9. Veți declara un app.get() general pentru calea "/", care tratează orice cerere de forma /pagina randând fișierul pagina.ejs (unde "pagina" e un nume generic și trebuie să funcționeze pentru orice string). Atenție, acest app.get() trebuie să fie ultimul în lista de app.get()-uri. Dacă pagina cerută nu există, se va randa o pagină specială de eroare 404 (în modul descris mai jos).
10. Se va da ca argument în funcția render o funcție callback function(eroare, rezultatRandare) care, în cazul în care mesajul erorii începe cu "Failed to lookup view" va afișa pagina pentru eroarea 404 (vezi mai jos). În cazul în care e altă eroare va afișa pagina de eroare generică (vezi mai jos), iar dacă nu sunt erori va trimite către client rezultatul randării.
11. Pentru randarea erorilor, veți folosi un fișier json, numit erori.json. Acesta va avea următoarele proprietăți:
 - a. **cale_baza**: calea la care se găsesc imaginile corespunzătoare erorilor
 - b. **eroare_default**: va fi un obiect JSON cu proprietățile: titlu (titlul paginii de eroare), text, imagine
 - c. **info_eroi**: un vector de obiecte. fiecare obiect descrie o eroare și are proprietățile: identificator (un cod numeric; pentru erorile http, precum 403, 404 e chiar codul http), status (boolean prin care indicăm dacă trebuie alt cod status decât 200

pentru răspuns), titlu (titlul erorii, pus în heading), text (descrierea erorii), imagine (o imagine descriptivă pentru eroare; se va da calea relativă la cale_baza). Se vor adăuga în proiect în calea specificată în cale_baza imagini pentru erorile definite.

12. Se va crea un template (eroare.ejs) cu ajutorul căruia să se afișeze erorile. Acesta va avea, preluate din locals, titlul, textul imaginii erorii.
13. Se va crea o variabilă globală numită obGlobal de tip obiect. Aceasta va avea o proprietate numită obErrori, implicit cu valoare null. Se va crea o funcție, numită **initErrori()**, fără parametri, care citește JSON-ul cu erorile și creează un obiect corespunzător lui cu toate datele erorilor (pentru a le avea încărcate în memorie), acest obiect va fi salvat în proprietatea obErrori a variabilei obGlobal. Pentru fiecare eroare, se va seta calea absolută în proprietatea imagine (folosind proprietatea cale_baza)
14. Se va crea o funcție de afișare a erorilor, numită **afisareEroare()** care va primi un obiect de tip Response, identificatorul, titlul, textul și imaginea erorii. În cazul în care există o eroare cu acel identificator, și titlul, textul și imaginea nu sunt precizate, se preiau datele încărcate din JSON pentru afișarea erorii. Dacă una dintre cele 3 proprietăți ale erorii e dată ca argument în funcție, are prioritate asupra datelor din JSON și se va afișa în pagină (de exemplu dacă titlul e dat ca argument, se afișează argumentul nu titlul citit din JSON). În cazul în care identificatorul nu se specifică, se afișează o pagină de eroare cu datele din eroare_default, însă tot cu posibilitatea de a afișa titlul, textul și imaginea din argumente, dacă sunt precizate.
15. Veți mai face încă o pagină (cu puțin text sau imagini, ca să aibă conținut), de exemplu o pagină cu descrierea site-ului sau istoricul său, al firmei virtuale pentru care este făcut etc. Această pagină trebuie să poată fi accesată prin meniu (linkul să fie corect și să transmită o cerere de tip get). Nu faceți încă pagina de produse, fiindcă pe acelea le preluăm din baza de date. Nici paginile de înregistrare sau login, fiindcă le tratăm separat.
16. În zona din layout de date despre utilizator vom afișa ip-ul utilizatorului (prin program). Deocamdată, site-ul fiind local, veți vedea mereu ip-ul de localhost (adică ::1). Ip-ul real se va vedea când adăugați site-ul pe Internet.
17. La o cerere către o cale din /resurse (de exemplu, localhost:8080/resurse/css/) fără fișier specificat (către folderul care ar conține fișierul) se va returna eroarea 403 Forbidden. Pagina de 403 va avea format similar cu cea de 404, folosind același template (eroare.ejs), dar textul și imaginea schimbate corespunzător, preluate din JSON
18. La cererea oricărui fișier cu extensia *ejs* se va transmite o eroare de tip 400 Bad Request. Pagina de 400 va folosi același template (eroare.ejs), dar textul și imaginea schimbate corespunzător, preluate din JSON
19. Să se adauge un `app.get()` pentru `"/favicon.ico"`. Uneori browserele cer favicon pentru diverse răspunsuri primite de la server (nu neapărat fișiere html, unde putem specifica faviconul prin taguri link). Pentru această cerere vom trimite un favicon cu metoda `sendFile()`.
20. Proiectul nostru va folosi niște foldere în care generează fișiere. Scrieți un vector cu numele folderelor de creat, numit `vect_foldere`, (vectorul va conține numele folderelor: "temp", "logs", "backup", "fișiere_uploadate"). Se va itera prin vector și se va testa dacă folderul există. Dacă un folder nu există, este creat. Peste tot unde aveți concatenare de căi folosiți `path.join()`.

(0.05) Task video:

1. Videoclipul va avea o imagine poster (atributul poster) relevantă pentru conținut. Videoclipul va avea și elementele de control afișate implicit. Videoclipul trebuie să se afișeze frumos pe ecran mic, deci va avea în css lățimea setată în procentaje, cât și o lățime minimă și maximă, ca să nu devină greu de urmărit pe ecrane mici sau foarte mari.
2. Videoclipul va avea mai multe trackuri (minim 2), în limbi diferite (de exemplu română și engleză) cu atributele setate conform limbii. Trackul în limba română va fi cel default. Trackurile trebuie să aibă măcar 3 mesaje.
3. Videoclipul va avea preîncărcate metainformațiile (preload).
4. Videoclipul va avea minim 2 surse: același videoclip dar în formate diferite. Puteți converti online videoclipul în mai multe formate.
5. Textul VTT asociat videoclipului va avea background de culoare c1 deschisă. Textul va fi scris cu o culoare c2 închisă, conform schemei cromatice. Pe parcursul videoclipului, o secvență de text din subtitrări va fi afișată în colțul dreapta sus a videoclipului. O secvență din subtitrări va conține un cuvânt subliniat.

Observație: realizați acest task abia după ce ați trecut site-ul pe node, altfel nu vă vor merge track-urile.

(0.05) Task linkuri:

Linkurile din pagini se vor stiliza conform următoarelor cerințe:

- Linkurile vizitate vor avea o culoare diferită de cea implicită, setată printr-o variabilă CSS, la una dintre culorile schemei cromatice alese.

- Linkurile externe vor avea generat cu ajutorul unui pseudoelement `::before` cu simbolul `⌂` cu font-size cu 50% mai mare față de dimensiunea fontului linkului (folosiți unități de măsură relative).
Indicație: putem selecta linkurile externe, selectând tagurile a cu atributul href a căru valoare începe cu "http".
- La aducerea cursorului pe un link extern, simbolul (generat prin css) își va schimba culoarea
- La aducerea cursorului pe un link din conținutul paginii (linkuri din main, nu din meniu) , o pata de culoare va crește din centru, așa cum se vede în videoclip, și se va micșora la loc când luăm cursorul de pe link. Pata de culoare s-a realizat folosind o imagine de background de tip gradient radial, având (în cadrul gradientului) atât la 0% cât și la 40% din gradient o culoare c1, iar la 80% și 100% culoarea *transparent*. Imaginea de background e poziționată centrat (background-position) și de dimensiune 0 (background-size). Când se vine cu cursorul pe link, imaginea de background crește la 100% cu ajutorul unei tranziții.



- La click pe un link din pagină (:active), va apărea un border în jurul lui așa cum se vede în videoclip.

(0.05) Adăugați în `.gitignore` folderul `node_modules` împreună cu cele create de aplicație (subpunctul 20)

Bonus:

Se va implementa o funcție de verificare a datelor din JSON-ul de erori, care va fi apelată la pornirea serverului. Funcția va afișa mesaje de eroare (cu text clar, detaliat și relevant care să explice problema, pentru a fi remediată) în următoare situații:

- (0.025) Nu există fișierul `erori.json` - caz în care, pe lângă afișarea mesajului, aplicația se va și închide (`process.exit()`)
- (0.025) Nu există una dintre proprietățile: `info_eroi`, `cale_baza`, `eroare_default`
- (0.025) Pentru eroarea default lipsește una dintre proprietățile: `titlu`, `text` sau `imagine`.
- (0.025) Folderul specificat în `"cale_baza"` nu există în sistemul de fișiere
- (0.05) Nu există (în sistemul de fișiere) vreunul dintre fișierele `imagine` care sunt asociate erorilor. Veți modifica json-ul de erori astfel încât fiecare eroare să aibă altă imagine.
- (0.2) Pentru un obiect din fișier există o proprietate specificată de mai multe ori (de exemplu apare de două ori proprietatea `titlu` (atenție verificarea trebuie făcută pe string, nu pe obiectul rezultat din fișier).
- (0.15) Există mai multe erori (în vectorul de erori) cu același identificator (în mesajul de eroare se vor preciza toate proprietățile erorilor, în afară de identificator, pentru a le găsi ușor)

Etapă 5	24.04.2026	27.04.2026	
---------	------------	------------	--

Etapă 5 (1.1p)

(0.35) Galeria statică:

Se va implementa o galerie statică. Galeria va fi vizibilă atât pe pagina principală cât și într-o pagină separată, dedicată ei (indicație: nu duplicați codul ci includeți un fragment care să conțină galeria), folosind `grid` și îndeplinind cerințele de mai jos:

- Detaliile pentru imaginile galeriei se vor lua dintr-un json cu următorul format:
 - Obiectul principal al json-ului va avea o proprietate numită `"cale_galerie"` cu calea către folderul în care se află toate imaginile galeriei
 - Obiectul principal al json-ului va avea și o proprietate `imagini`, care va fi de tip vector și va avea ca elemente obiecte în formatul următor:

```

{

    "cale_fisier":"image.png",

    "titlu":"Nume imagine",

    "alt":"text alternativ imagine", "text_descriere":"text descriere",

    "luni":["ianuarie","februarie","iunie"]

}

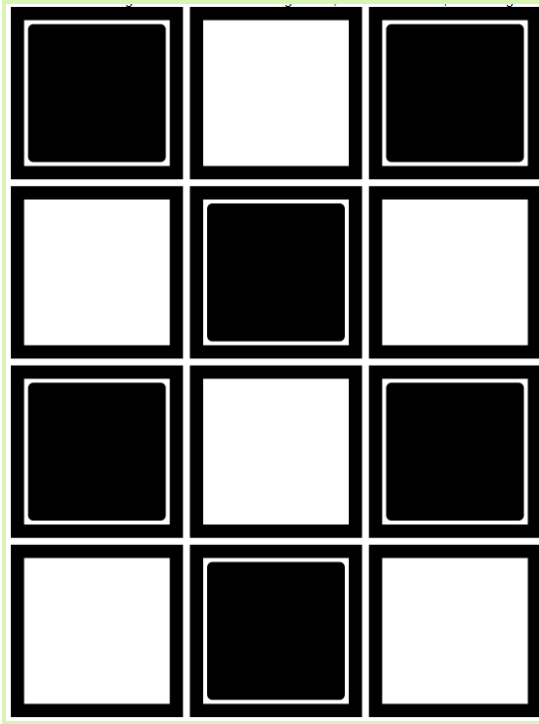
```

- aceste detalii se vor transmite către fișierul EJS și vor fi folosite în template-ul de afișare a imaginilor.
- Serverul nu va transmite toate imaginile către pagină, ci doar cele care îndeplinesc un anumit criteriu. Imaginile se vor afișa în funcție de luna (preluată din data curentă). De exemplu, dacă data curentă se încadrează în una din lunile decembrie, ianuari sau februarie, se va afișa imaginea din exemplu.
- Imaginile vor avea toate aceleași dimensiuni. Acest lucru se poate realiza folosind aproape orice utilitar grafic, ajutându-vă de funcționalitățile *crop*(decupare) și *resize*(redimensionare). Un tool online care e destul de ușor de folosit: <https://picresize.com/en/edit>. Imaginile vor fi editate astfel încât să aibă o dimensiune pentru ecran mare (de exemplu o lățime de 400-500px).
- Imaginile vor avea toate câte un nume sau descriere asociată (vizibilă) cu ajutorul tagurilor `<figure>` și `<figcaption>`. **Imaginile vor fi indexate, iar indexul va fi scris la începutul descrierii din figcaption. Indexarea se va face cu litere mici grecești (α , β , γ , ...) (ideal folosind un *counter* css).** Imaginile vor fi puse într-un tag `picture`, în care vor mai exista surse pentru ecran mediu și mic. **Se vor genera imagini de dimensiune mai mică (dacă nu există deja) prin node** (atunci când un client cere pagina). Indicație: pachetul `sharp`. Descrierea imaginilor se va adăuga în atributul `title`. În proprietatea `alt` a imaginilor se va adăuga valoarea proprietății `alt` din JSON, sau dacă aceasta nu există în formatul primit, se va adăuga numele imaginii.
- Galeria se va întinde pe toată lățimea paginii site-ului (în sensul că nu vor fi elemente în stânga sau dreapta, ci doar eventuale spațieri).
- Forma galeriei e descrisă mai jos (zonele cu negru reprezintă imagini iar cele cu alb, locuri libere în grid). Veți folosi un grid pentru a o afișa:
 - În cazul în care mai multe imagini se potrivesc criteriului de afișare, numărul de imagini afișate se va trunchia la 12 (acest lucru trebuie implementat în program, nu obținut doar din date).
 - Pe ecran mare imaginile se vor aseza într-un grid $n \times 3$, cu n divizibil cu 2, ca în imaginea de mai jos, folosind grid:
 - Pe ecran mediu, imaginile se vor aseza pe două coloane, folosind grid.
 - Pe ecran mic, imaginile se vor aseza una sub alta, pe o coloană.
 - **Tranziție.** Când se vine cu cursorul pe o imagine din galerie, se va realiza următoarea tranziție: textul din `figcaption`, asociat imaginii, se va scala treptat (fără a se scala și imaginea), simetric față de centrul boxului descrierii, iar imaginii îi va crește treptat contrastul la 200% (proprietatea `filter`). Tranziția va dura o secundă și 100 milisecunde.
 - Imaginile folosite trebuie să aibă o licență corespunzătoare (să nu fie folosite ilegal). Puteți folosi imagini din "public domain" (domeniul public), licențiate CC0 (creative commons 0). Totuși pentru a învăța cum să inserați o atribuire de autor pentru o imagine folosită, minim una dintre ele va fi luată de pe un site care o oferă cu licență CC-BY (necesită atribuire). Datele pentru atribuire vor fi salvate în JSON.

Observație: nu e nevoie să folosiți aceleași culori, fonturi, dimensiuni de text, stiluri de scris etc cu cele date în cerință. Importantă e forma galeriei și tranziția pe fiecare imagine.

Observație: Pentru verificarea afișării imaginilor în funcție de timp veți schimba valoarea variabilei de tip dată care preia data și ora curentă, la o dată și o oră care să arate schimbarea de afișare.

Puteți, de asemenea, să îmbogățiți galeria cu propriile voastre idei dar doar adăugând funcționalități, și nu modificând cerința de bază. **Se va da un bonus pentru creativitate** în cazul unor astfel de adăugări.



(0.25) **Compilare automată scss.** Se vor realiza următoarele subpuncte:

- Pregătire cadru de lucru.** Se vor defini în obiectul global două proprietăți *folderScss* și *folderCss* care conțin căile din folderul de resurse (depinzând de `__dirname`). Se va adăuga folderul *backup* la lista folderelor create automat de aplicație (așa cum e și folderul *temp*)
- Funcția de compilare a scss-urilor.** Se va face o funcție `compileazaScss(caleScss, caleCss){}` care compilează un fișier scss în fișier css. Primii 2 parametri reprezintă căile către fișierul scss (inputul funcției) și fișierul css (outputul funcției). Dacă avem căi absolute se iau fișierele de la cele două căi, iar dacă sunt relative se vor considera relative la *folderScss*, respectiv *folderCss*. compilarea se va face cu ajutorul pachetului sass. Dacă numele/calea fișierului css lipsește, se va salva în *folderCss* rezultatul compilării folosind numele fișierului scss, dar cu extensia *css*
- Salvare în backup.** În cadrul funcției `compileazaScss`, înainte de compilarea automată a scss-ului în fișierul css asociat, fișierul css vechi cu același nume va fi copiat în subcalea *resurse/css* a folderului *backup*. Orice folder din această subcale va fi creat dacă nu există deja. Se va afișa un mesaj de eroare în cazul eșecului copierii.
- Compilare inițială.** La pornirea serverului, toate fișierele scss din *folderScss* trebuie să fie compilate în fișierele css cu același nume folosind funcția `compileazaScss`. Înainte de suprascrierea fișierului css, acesta va fi copiat în folderul *backup* (suprascriind un backup cu același nume - sau dacă vreți să păstrați backup-urile anterioare puteți integra în nume o informație cu privire la timpul creării).
- Compilare pe parcurs.** Se va scrie cod (folosind `fs.watch()`) astfel încât să se urmărească modificările din folderul de fișiere scss. La modificarea/crearea unui fișier acesta va fi compilat automat în css. Fișierul css va avea același nume cu fișierul scss, având doar extensia scss schimbată în css. Înainte de suprascrierea fișierului css, acesta va fi copiat în folderul *backup* (suprascriind un backup cu același nume - sau dacă vreți să păstrați backup-urile anterioare puteți integra în nume o informație cu privire la timpul creării).

(0.25) **Customizare Bootstrap** cu schema cromatică aleasă de voi și cu dimensiuni de ecran diferite pentru ecrane medii și mari. Veți face un fișier numit `custom.scss` în care folosiți fișierul scss al bootstrap, schimbând valori pentru:

- culorile de background pentru minim 2 teme la alegere pe care aplicați tema cromatică schimbată (customizată de voi)

- culori de font (adică ale literelor. **Precizare:** nu neaparat pentru toata pagina, orice culoare de litere - de exemplu dintr-un buton)
- dimensiuni de ecran diferite pentru ecrane medii și mari
- dimensiunea razelor de border
- dimensiunea literelor headingurilor (h1,h2 etc)
- familia de font implicită
- încă una sau mai multe variabile alese de voi

Corectare Bootstrap. Atenție, este posibil ca integrarea bootstrap să afecteze aspectul site-ului deoarece pentru anumite elemente din pagină v-ați bazat pe aspectul implicit al acestora, prin urmare va fi nevoie să definiți reguli css astfel încât site-ul să revină la forma inițială. CSS-ul pentru bootstrap trebuie pus primul pentru a asigura suprascrierea proprietăților pentru selectorii-tag.

Veți folosi unul sau mai multe elemente de bootstrap care să ilustreze schimbările, dintre cele de mai jos:

Customizarea se va face ca la laborator folosind sass, iar **compilarea se va face automat la repornirea serverului folosind funcția compileazaScss!**

(0.25) (Efecte CSS)

Observație: având în vedere că depășirea punctajului pentru lista de efecte de mai jos se transformă în bonus adunat la proiect, aceasta lista poate fi extinsă pe tot parcursul semestrului (deci inclusiv după deadline-ul etapei 4), adăugând noi idei pentru bonusuri, deoarece bonusurile pot fi prezentate și mai târziu. **Atenție! Anumite efecte au cerință individuală asociată (care trebuie respectată, pentru a fi punctate) - enunțurile acestora au identificatorul prefixul "efect_css".** Cele care nu au textul "cerința individuală" au enunțul întreg în acest fișier.

Efecte css propuse (se vor **alege** efecte css de implementat ca să **însuneze 0.25** - nu e obligatoriu să le faceți pe toate din listă)

• (0.05) **Duotone**

În cadrul site-ului va apărea o imagine (de fapt un div cu background-ul egal cu acea imagine) pe care s-a aplicat efectul duotone (pentru culorile c1 și c2) folosind pseudoelementele before și after și proprietatea mix-blend-mode. Inițial imaginea apare fără efect (cele două pseudoelemente asociate culorilor c1 și c2 din duotone fiind ascunse în stânga și în dreapta elementului imagine). Când se vine cu cursorul pe element, cele două pseudoelemente vin din părți suprapunându-se cu elementul imagine și formând imaginea duotone. când se ia cursorul de pe imagine, imaginea revine la forma inițială.

• (0.15) **Reflexie**

Să se aleagă un text, de exemplu, un heading, pe care să se aplice reflexia. Se poate folosi [exemplul de reflexie prin dublarea elementului](#), sau se pot folosi proprietăți precum box-reflect însă trebuie să găsiți metode prin care reflexia e vizibilă pe orice browser, măcar cele mai noi versiuni (varianta din exemplu este portabilă).

Modul de aplicare al reflexiei trebuie să fie următorul:

- Reflexia trebuie să fie în dreapta, reflectată față de axa OY
- Când se vine cu cursorul pe text sau reflexie, atât textul cât și reflexia, își schimbă culoarea



- (0.025) **Scrierea textului pe coloane**, folosind proprietatea column-count. Se va alege o secțiune cu mai mult text pe care se va aplica proprietatea column-count. Pe ecran mic și mediu se va afișa o singură coloană. Între coloane se va afișa și o linie despărțitoare (column-rule)

- (0.025) **Schimbarea afișării implicite a textului selectat**, folosind pseudo-clasa `::selection` - schimbați minim 2 proprietăți ale textului selectat, folosind variabilele definite pentru schema cromatică.
- (0.05) **text care se plimba** orizontal sau vertical printr-o animație (keyframes) recurentă (după ce se termină mesajul, reapare). Elementul cu textul trebuie să fie responsive (să nu apară scrollbar orizontal pe pagină din cauza lui).
- (0.05) **background fix la scroll** într-una din pagini, folosind background-attachment. Imaginea de background se va schimba (printr-o animație) după t secunde (t e ales de voi).
- (0.05) **Afișarea unui tabel astfel încât să fie responsive**, conform [exemplului dat](#). Tabelul ales trebuie să aibă minim 4 coloane și să nu conțină celule cu `rowspan/colspan`.
- (0.025) **Afișarea unui tabel transpus** pe o dimensiune de ecran (media query) conform [exemplului](#). Pe restul dimensiunilor de ecran se va afișa la fel.
- (0.1) **Stilizare hr**
Adăugați în pagină un `<hr>` (de exemplu între două secțiuni, două paragrafe cu teme diferite, două anunțuri). Stilizați hr-ul astfel încât să semene cu imaginea de mai jos. **Atenție! Nu folosiți culorile din imagine ci culori din schema cromatică aleasă de voi. dimesniunile hr-ului sunt la alegerea voastră (atâta timp cât e totuși vizibil)**
Indicații de rezolvare. Stelutele se generează cu `::before` și `::after` și se poziționează (de exemplu cu poziție nestatică și proprietățile `left` și `right`, sau cu `margin-left`, `margin-right`. Puteti pune în loc de stelute alte caractere sau chiar imagini.
- (0.05) **videoclip care se comporta ca un background**, conform exemplului de la <https://css-tricks.com/full-page-background-video-styles/>

Bonus 1:

(0.5) Galeria animata:

Realizați o galerie de imagini cu un număr aleator de poze, divizibil cu 3, mai mic decât 16, conform videoclipului de mai jos. La fiecare încărcare a paginii se va genera un nou număr de imagini. Imaginile se vor încărca din JSON-ul de imagini pentru galerie statică. Imaginile alese vor fi consecutive în JSON și vor avea un offset ales aleator prin program (de exemplu, dacă offset-ul e 5, imaginile vor fi cu indicii 5, 6, 7...etc).

Imaginile din galerie trebuie să fie distincte.

Galeria va avea un border-image cu o imagine aleasă de voi. Folosiți SASS în realizarea galeriei. Observație: CSS-ul va trebui generat prin node pe baza fișierului SASS ca să acomodeze noul număr de imagini.

Trecerea la următoarea imagine se face prin decuparea imaginii curente (clip-path) începând din centru, și micșorând zona vizibilă spre margini, atât în stânga cât și în dreapta.

Galeria va relua imaginile după ce o afișează pe ultima, rezultând într-o afișare continuă, repetitivă. **Când se vine cu cursorul pe galerie, animația se oprește. Observați pauzele pe care le face în videoclip când cursorul este pe galerie.** Galeria animată nu se va afișa pe ecran mediu și mic.

Bonus 2:

Se poate considera ca bonus să faceți mai multe efecte css care ar depăși cele 0.25 puncte.

Bonus 3:

(0.05) Fișierele salvate în backup (în funcția `compileazaScss`, în urma compilării `scss -> css`) să aibă în nume o informație de timp (de exemplu, în loc de `a.css` să fie fișierul `a_timestamp.css`, de exemplu `a_1681124489791.css`) pentru a putea salva mai multe versiuni ale aceluiași fișier.

Bonus 4:

(0.025) În cadrul codului dat la curs, pentru compilarea fișierelor `scss`, aceasta funcționează corect doar dacă în numele fișierelor nu există puncte (de exemplu nu ar merge pentru `stil.frumos.css`). Corectați codul astfel încât să funcționeze și pentru acest tip de fișiere.

Bonus 5:

Se va implementa o funcție de verificare a datelor din JSON-ul cu imagini, care va fi apelată la pornirea serverului. Funcția va afișa mesaje de eroare (cu text clar, detaliat și relevant care să explice problema, pentru a fi remediată) în următoarele situații:

- a. (0.025) Folderul specificat în "cale_galerie" nu există în sistemul de fișiere
- b. (0.025) Nu există (în sistemul de fișiere) vreunul dintre fișierele imagine specificate în lista de imagini.

Etapă 6	22.05.2026	29.05.2026	
----------------	------------	------------	--

Taskuri etapa 6 (baza de date + JavaScript introductiv) - afișarea produselor (punctaj recomandat 1.75p)

(0.05) corectarea linkurilor din meniu; **daca nu s-a facut deja acest lucru.**

(1.2p) afisare + sortare/filtrare/calculare

Aplicatie de sortare, filtrare si calculare

1. Descriere cerință. Pagină produse.

Va exista o pagina de produse in care trebuie listate datele pentru anumite produse/entitati specifice site-ului (le vom numi de acum inainte "entitatile aplicatiei"). Entitatile aplicatiei vor fi, de exemplu, carti pentru o librerie sau un site de lectii online, tipuri de prajituri si fursecuri pentru o cofetarie, flori si buchete pentru o florarie etc. - **dacă nu aveți idei de produse, îmi dați mesaj și ne gândim împreună.** Tipul de entitate trebuie sa fie strict legat de tema site-ului. În pagina de produse **nu** se vor afișa toate detaliile produselor ci doar cele relevante pentru filtrare/sortare (ideal cele mai importante detalii, precum specificații des căutate de către utilizatori).

2. Descriere cerință. Pagină produs unic..

Se va genera automat prin program câte o pagină pentru fiecare produs (conform exemplului dat la curs si/sau laborator). Pagina respectivă va afișa toate detaliile produsului (inclusiv cele care nu apar in pagina de produse). Pentru asta, se trimite prin intermediul obiectului *locals(ejs)* datele produsului.

3. Baza de date. Tabelul de produse

Se va crea o bază de date pentru proiect. Se va crea un tabel de produse. Se va crea un utilizator (al bazei de date), folosit în program, pentru care se alocă drepturi pe baza de date și tabel. Datele cuprinzând proprietățile produselor (pentru afișarea în pagină) se vor prelua din tabel. Produsele trebuie sa contina minim urmatoarele proprietati (pentru a avea cu ce să lucrăm în aplicația de sortare/filtrare):

1. id (identificatorul unic, din json; id-ul trebuie sa fie numeric)
2. nume
3. descriere
4. o imagine (in tabel se va salva doar calea catre imagine)
5. categorie mare (o clasificare a entitatilor din date, de exemplu, daca e vorba de pisici, sa fie clasificate dupa rasa sau talie, daca e vorba de carti, dupa tematica, daca e vorba de calculatoare: de birou, de gaming etc). Categoria se va face cu ajutorul unei enumeratii, nu va fi de tip string. Enumeratia ar trebui să aibă maxim 5 valori.
6. Un mod de categorizare mai puțin important (de exemplu, pentru pisici: după culoare sau după lungimea blanei, pentru cărți: o subtematică sau tipul de hârtie folosit (carte cartonată, hârtie simplă, hârtie lucioasă, carte cu învelitoare), sau tipul de expediere posibilă (curier, de la centrul de distribuție, poștă), vârsta pentru care e indicat produsul respectiv (copii, adolescenți, adulți), pentru flori, de exemplu, pot fi modurile de prezentare(floare singură, buchet, ghiveci, floare presată etc).
7. pret (sau alta caracteristica cu valoare numerica)
8. o a doua caracteristică numerică (de exemplu - pentru diverse produse- poate fi: dimensiune, volum, o specificație tehnică precum putere sau tensiune, numărul de pagini pentru o carte, numărul de caractere/randuri/imagini/alte elemente care apar pe o felicitare, numărul de culori, timpul de garanție, gramaj, număr de calorii pentru un produs alimentar etc)
9. caracteristica cu valoare de tip Date (data calendaristica) - de exemplu poate fi data de când există produsul în oferta magazinului.
10. caracteristica care poate sa aiba doar o singura valoare pentru o entitate (dint-un set de valori - de exemplu, culoarea)
11. caracteristica care poate sa aiba mai multe valori pentru o entitate (poate fi folosit intr-un select de tip multiplu; in tabel valorile vor fi puse intr-un singur camp si vor fi separate cu virgula, de exemplu pentru un produs alimentar, campul ar putea fi ingrediente cu valoarea "sare, faina, lapte, branza").
12. caracteristica booleana (de exemplu, are sau nu are o anumită caracteristică, poate sau nu să fie expediat prin poștă, admite sau nu voucher; exemple pentru diverse proprietăți: pentru cărți: include sau nu semn de carte; pentru prăjituri: bun sau nu pentru bolnavii de diabet, pentru fast-food, include sau nu un anumit alergen, pentru imagini/felicitări e color sau nu, etc.)

4. Dacă nu aveți idei pentru o anumită caracteristică potrivită pentru produsele site-ului vostru, îmi dați mesaj și discutăm ce s-ar potrivi mai bine.

Se vor introduce în tabel 15-20 entitati cu caracteristici suficient de diversificate (pentru a putea verifica toate tipurile de filtre si sortari)

5. Meniu. Împărțirea pe categorii mari

În meniu se va adăuga o opțiune nouă numită "Produse". Opțiunea va avea următoare subopțiuni: toate (în sensul că se afișează toate produsele), *categ_1*, *categ_2*, ..., *categ_n* (acestea sunt numele valorilor categoriei mari). **Valorile categoriei mari, care apar în meniu trebuie generate pe baza enumerăției din baza de date (nu se scriu manual ci se generează prin program și se**

transmit în meniu prin locals). La click pe o astfel de subopțiune, se vor afișa în pagină doar produsele corespunzătoare acelei categorii. Atenție, filtrarea se va face aici la nivel de server, în sensul că vectorul de produse trimis de server spre afișare conține doar produsele din categoria cerută din meniu. Toate subopțiunile vor face același apel get, dar cu parametri diferiți (cu alte cuvinte folosiți același ejs pentru afișarea tuturor produselor cât și afișarea pe categorii).

6. **Format afișare produse (pentru pagina de produse).**

Produsele se vor afișa prin ejs, cu ajutorul unui for care populează un template cu informațiile fiecărui element din fisierul de date. La început se vor afișa toate produsele, dar prin completarea unor inputuri de filtrare se vor păstra doar o parte din produse în pagină (cele care nu corespund filtrului setat doar vor fi ascunse, nu șterse din DOM, pentru a le putea folosi și în filtrări ulterioare).

Formatul template-ului e cel precizat mai jos:

Va exista un element de tip article care va avea un heading de nivel corespunzător, cu numele entității.

Restul afișării se va împărți pe două coloane

Prima coloană va conține imaginea. Pentru estetica, imaginile ar trebui să fie de aceeași dimensiune și relativ mici ca să încapă în formtul de afișare.

Tot în prima coloană, vor fi afișate una sub alta încadrul unei liste (deci fiecare caracteristică pe un rând separat):

1. caracteristică numerică
2. caracteristică cu o singură valoare string
3. caracteristică cu mai multe valori string
4. caracteristică dată
5. caracteristică booleană

7. Fiecare caracteristică va fi precizată printr-o pereche de forma nume_caracteristică: valoare_caracteristică. Numele caracteristicii va fi bold și cu o culoare de background sau de text diferită de cea a valorii.

Data calendaristică se va scrie cu ajutorul tagului <time>. Conținutul tagului va fi data în limba română în formatul: 'zi nume_luna an [zi_săptămână]', de exemplu, 15 Septembrie 2018 [Sambătă]

A doua coloană va conține categoria entității. De asemenea articolul cuprinzând entitatea va trebui să aibă printre eventuale alte clase (atributul class) și numele categoriei (deci categoria trebuie să nu cuprindă spații)

Sub categorie (tot în a doua coloană) va fi afișată și descrierea.

Id-ul fiecărui articol este de forma "artc-" + id, unde id e id-ul efectiv din fisierul de date (deci id-uri posibile ar fi: artc-2, artc-7, ...)

Puteți adăuga alte elemente de stilizare dar fără a modifica cerința. Dacă aveți nevoie de date în plus pentru filtrare/sortare le puteți include în format, însă e important să nu apară toate detaliile produsului, acestea fiind rezervate pentru pagina proprie produsului.

8. **Filtre. Inputuri**

Va exista și o secțiune (care să nu fie formă chiar dacă conține inputuri) deasupra zonei de afișare a produselor, care va conține următoarele tipuri de input, fiecare corespunzând unui câmp dintre caracteristicile entităților/produselor). Toate inputurile vor avea și o etichetă asociată (label). **Pentru anumite inputuri din lista de mai jos veți primi cerințe specifice la subpunctul următor (deci le veți integra în acele cerințe, nu folosiți același tip de input de 2 ori); restul de inputuri vor avea rol decis de voi în filtrare.**

1. input de tip text
2. input de tip range. Pentru inputul de tip range trebuie să apară valoarea minimă în stânga lui și valoarea maximă în dreapta lui, și de asemenea să se afișeze valoarea selectată (de exemplu în paranteză, după valoarea maximă).
3. datalist
4. grup de inputuri de tip radio (va avea și o opțiune generală care să nu aplice niciun filtru)
5. input de tip checkbox sau grup de inputuri de tip checkbox (în funcție de tipul de filtru)
6. textarea
7. select simplu (prima opțiune va fi și cea default (initial selectată), cu textul "nimic" sau "oricare" sau ceva asemănător care să se refere la situația în care acel filtru nu ne interesează).
8. select multiplu

9. Valorile inițiale ale inputurilor vor fi de așa natură încât să se potrivească cu toate produsele. De exemplu dacă sunt mai multe tipuri de culoare pentru produs și folosim un grup de radiobuttons pentru ele, să existe și un radiobutton pentru opțiunea "orice culoare" care va fi implicit selectat.

10. **Cerințe specifice filtre.**

Produsele se vor afișa fie cu ajutorul unui grid, fie folosind flexbox.

1. Inputul de tip text va fi pentru numele produsului. Inputul de tip text acceptă și un nume greșit dacă nu diferă decât cu maxim 2 litere față de numele real. De exemplu dacă întorducem "saavrină" ne va da produsul "savarină"
2. Se va face un grup de checkbox fiecare având în dreptul său și un grup de radiobuttons cu valorile "are", "nu are" (sau alte etichete mai relevante). Grupul de checkbox-uri va fi dedicat caracteristicii care poate să aibă mai multe valori. Se vor afișa doar produsele care au printre valorile multiple cele selectate în checkbox cu radiobuttonul setat pe valoarea "are" și care în același timp nu au printre valori o valoare din checkboxurile cu radiobutton-ul "nu are" setat. Valorile din checkboxurile nebifate nu participă la filtrare. Implicit toate checkboxurile sunt nebifate și în fiecare grup de radiobuttons e setată valoarea "are"

11. Pentru filtrele/inputurile pentru care nu s-a precizat cum anume să fie implementate în pagină, alegeți voi pentru ce caracteristici să se aplice și în ce manieră. Nu e voie să aveți două filtre pentru aceeași proprietate a produsului.

12. **Butoane. Operațiunile efective de filtrare/sortare/calculare**

Alături de filtre vor exista și următoarele butoane:

1. Un buton cu textul "filtreaza" care va filtra entitatile dupa **toate** inputurile activate.
 2. Doua butoane de sortare (unul pentru ascendent si unul pentru descendent). Sortarea se va face dupa doua chei, de exemplu daca se sorteaza după cheile c1 și c2, intai se va sorta după c1, si pentru valori egale pentru c1, se va sorta după c2. **Cele două chei după care trebuie să sortați** sunt, în ordine: raportul caracteristica_numerica_secundara/preț și subcategoria(categoria de mai mică importanță)
 3. Un buton de calculare (de exemplu, calculeaza suma/media/minimul/maximul tuturor preturilor sau doar al prețurilor elementelor selectate). Calculul va aparea într-un div cu poziție fixă, care va fi creat dinamic (createElement), va sta pe ecran 2 secunde și apoi va dispărea (va fi șters).
 4. Un buton de resetare a filtrelor.
13. (Specificatii pentru anul 2 CTI) La click pe butonul de resetare, printr-un mesaj de tip "confirm" utilizatorul va fi intrebat daca vrea cu adevarat sa reseteze filtrele. Daca raspunde "ok", toate filtrele vor reveni la valorile implicite care corespund afișării tuturor produselor. Se reafixează toate produsele (fără niciun filtru aplicat) și în ordinea inițială (resetarea anulează și sortarea).
14. [Edit: era inainte paragraf, l-am facut item de lista pentru claritate; enuntul e acelasi pentru anii 2,3,4] La click pe oricare dintre butoanele de filtrare/sortare/calculare se va verifica intai ca inputurile afectate de ele au valori valide (în special inputul de tip text și textarea-ul), si numai daca sunt valide se va realiza operatia. Daca valorile nu sunt valide, se afiseaza un mesaj de atentionare relevant. In cazul in care nu e specificat clar in cerintele de mai jos cum sa fie acest mesaj, studentul poate alege sa il afiseze cum doreste, astfel incat sa fie bine integrat in aplicatie (de exemplu un alert sau marcarea cu rosu a inputului cu probleme). Studentul va alege care sunt cazurile de input invalid (de exemplu in inputul de tip text a fost introdusa o cifra, si nu are sens pentru aplicatie, sau nu a fost selectat niciun radiobutton, sau inputul de tip textarea e vid etc).

(0.3p - fiecare e 0.05) Stilizare inputuri . Inputurile de pe pagina de produse se vor stiliza cu ajutorul Bootstrap, folosind customizari facute de voi in sass (vezi etapa 5). Puteti sa mai adaugati variabile in fisierul de customizare, care sa ajute la stilizarea butoanelor si inputurilor.

- a. **Butoanele (de filtrare, sortare, resetare) trebuie stilizate cu ajutorul temei bootstrap pentru care ati schimbat culoarea** (de exemplu primary sau secondary). **Raza si grosimea borderului** trebuie date de voi in fisierul sass de customizare prin variabile (daca nu ati facut asta deja). Butoanele si inputurile **vor fi dimensionate cu ajutorul claselor bootstrap**. Butoanele trebuie sa aiba iconuri (glyphicons) **relevante din bootstrap**: <https://icons.getbootstrap.com/>. Pe ecran mic se vor afisa doar iconurile fara text (realizati acest task in mod cat mai eficient - scris cat mai putin)
- b. Inputul de tip textarea va avea un **floating label** (bootstrap). In cazul validarii esuate a valorii din textarea (vezi cerinta cu validarea din etapa 5), floating label-ul va fi de tip **is-invalid** (se va seta prin javascript) si se va corecta automat daca valoarea din textarea devine valida.
- c. Fie inputurile de tip checkbox fie cele radio vor fi stilizate ca **toggle buttons**(din bootstrap). Butoanele deslectate trebuie sa fie desenate in stil outline (adica sa apara doar granita, fara background si textul sa fie colorat), iar la selectare sa apara cu background colorat - vezi clasele **btn-outline**. **Atentie trebuie folosit bootstrap nu css!**
- d. **Asezarea inputurilor** in mod ordonat si aliniat pe coloane in cadrul paginii de produse se va face printr-un **grid bootstrap** (vezi clasele row si col, impreuna cu clasele asociate query-urilor media).
- e. **Butonul pentru schimbarea temei sa fie un switch** din bootstrap (imageasau iconul cu luna/soarele, ramane in pagina, cu acelasi comportament) **Observatie pt cei care au bonusul cu mai multe teme: puteti inlocui cu alt element de bootstrap.**
- f. **Pentru inputul de tip range**,schimbati cu ajutorul customizarii bootstrap (prin variabile scss) dimensiunea bulinei care gliseaza (sa fie cu 50% mai mare fata de dimensiunea font-size-ului html-ului), schimbati de asemenea culoarea bulinei si culoarea sliderului. Variabilele necesare se gasesc in reference: <https://getbootstrap.com/docs/5.0/forms/range/>

(0.2p) light/dark theme cu variabile CSS (optional in SASS). Pentru moment va exista un buton în pagină care va face schimbarea. Butonul va fi reprezentat printr-o imagine cu soare (pt light) vs lună (pt dark) - poate fi un icon din fontawesome. Tema aleasă se va memora in localStorage si se va pastra tema la urmatoarea intrare pe pagina si pe restul paginilor site-ului.

Bonus 1- recomandat - este foarte usor de facut (se da 0.1p pentru fiecare tip distinct de input ajungand la maxim 0.8 puncte pentru cele 8 tipuri de input cerute in enunt) daca se genereaza attributele si eventual eticheta(label) **pe baza informatiilor din tabele si in general din baza de date** (precum select-ul facut impreuna la curs) . De exemplu daca avem un range pentru pret, putem pune ca valoare minima (atributul min) pretul minim din cadrul tabelului si ca valoare maxima (atributul max), pretul maxim.

Bonus 2 (0.15p) Utilizatorul să poată alege dintre 3 sau mai multe teme (nu doar light/dark). Ca și în cazul light/dark, tema aleasă se memorează în localStorage. Modul de alegere a temei se poate realiza printr-un select simplu sau radio buttons.

Bonus 3 (0.05p) Dacă nu sunt produse de afișat în urma filtrării, se va afișa un mesaj corespunzător în locul produselor (de exemplu: "nu exista produse conform filtrării curente").

Bonus 4 (onchange; 0.05 pentru fiecare filtru din cele 8 filtre pe care se aplica efectul - deci poate ajunge la maxim 0.4p). Atentie! Punctarea acestui bonus este conditionata de buna functionare a filtrarii si rezolvarea corecta si completa a cerintelor 6 si 7 din taskul cu produsele! Efectul de filtrare sa se intample la onchange (imediat la schimbarea valorii inputului) in loc sa se fac click pe butonul de filtrare.

Bonus 5 (0.5) Introduceți un sistem de paginare, fie in pagina de produse, fie in cea cu lista de utilizatori, fie in pagina de comenzi. Se va alege un numar fix, K, de elemente de afisat pe pagina. In cazul in care numarul de elemente de afisat este N, cu $N > K$ se vor genera $NRL = \lceil N/K \rceil$ linkuri, unde cu $\lceil \text{numar} \rceil$ s-a notat partea intreaga superioara a numarului dintre parantezele drepte (de exemplu, $\lceil 2.1 \rceil = 3$). Cele NRL linkuri vor fi numerele de la 1 până la NRL. La click pe un astfel de număr (să-l notăm P) se afișează în pagină produsele cu număr de ordine între $(P-1)*K$ și $P*K-1$ (considerând că produsele sunt indexate de la 0).

În loc de linkuri către pagini, puteți folosi și butoane sau select-uri.

Paginarea se poate face fie prin javascript client (se trimit toate produsele dar sunt vizibile doar cele din intervalul corect, fie prin cereri către server care returnează produsele cu indicii între limitele date.

Bonus 6 (0.5=0.2+0.1+0.2) Se vor adăuga trei butoane (cu iconuri, fără text, dar care să simbolizeze rolurile lor) pentru fiecare produs afișat. Când se vine cu cursorul pe fiecare buton se afișează un text explicativ cu rolul lor.

- primul buton va avea rolul de a păstra produsul asociat, chiar dacă o nouă filtrare teoretic l-ar șterge (un fel de cerere de a-l păstra pe ecran tot timpul). La click pe acest buton, atât produsul cât și butonul își schimbă stilul. Produsul nu mai dispăre la filtrare. Dacă se face din nou click pe buton, se dezactivează opțiunea, butonul și produsul revin la stilul inițial și produsul va putea fi din nou ascuns prin filtrare.

- al doilea buton va avea rolul de a șterge temporar produsul asociat, din afișarea curentă (utilizatorul e sigur că nu dorește acel produs).

Acesta poate să reapară la o nouă filtrare, dacă se potrivesc caracteristicile lui, sau o nouă sortare/resetare..

-al treilea buton va avea rolul de a șterge produsul din listă **pe timpul sesiunii curente**. Pentru acel tab în care e deschisă pagina, nu se mai afișează produsul în urma filtrărilor/sortărilor/resetărilor. Această proprietate se păstrează și la refresh sau la plecarea + revenirea în pagină, **dar doar în cadrul tab-ului respectiv**. La deschiderea paginii în alt tab, produsul apare normal. După închiderea tabului pe care s-a făcut setarea, se pierde compet setarea. (Hint: sessionStorage)

Bonus 7 (0.15p) Pentru inputurile de tip text sau elementele de tip textarea care impun căutarea unui cuvânt în cadrul numelui, descrierii sau a altor informații de tip text, simbolurile cu diacritice să fie văzute echivalent cu simburile fără diacritice. De exemplu, dacă în inputul pentru nume căutăm textul "briose" să primim produsul "brișe". O idee de rezolvare poate fi prelucrearea textului înainte de comparație, prin înlocuirea simbolurilor cu diacritice.

Bonus 8 (0.35p) Utilizatorul să poată alege cele două chei de sortare dorite dintr-un set cu minim 3 valori. Pentru fiecare cheie de sortare va fi un select (sau un alt tip de input care oferă o listă de valori din care se poate alege). Utilizatorul va alege din liste cheile de sortare și apoi va alege dacă vrea sortare crescătoare sau descrescătoare. De exmplu, poate sorta după nume și preț, apoi alege după preț și categorie etc.

Bonus 9 (0.5p) fiecare produs să aibă imagini multiple. Pe pagina proprie produsului se va afișa imaginea principală, iar cu ajutorul unor butoane (sau cu un carusel bootstrap) se va putea trece la imaginea următoare, schimbând imaginea curent afișată. Idee de rezolvare: în baza de date se memorează folderul în care se găsesc toate imaginile produsului (poate să nu fie neaparat folderul, dar din datele produsului programul să poată deduce unde sunt imaginile corespunzătoare lui).

Bonus 10a (0.5p=0.3 filtrare+0.2 sortare după 2 chei crescător/descrescător) **Acest bonus se punctează doar dacă există deja filtrarea și sortarea în JavaScript client.** Să se realizeze sortarea și filtrarea produselor la nivel de server, punând inputurile într-un formular. La click pe butonul de submit, se vor trimite datele din formular către server și serverul va întoarce listă de produse dorită. Pentru această cerință se poate face o pagină separată sau se poate încadra armonios în pagina de produse deja existentă.

Bonus 10b (+0.1p) Punctele pe acest bonus se adună la cele date pe rezolvarea cerinței de mai sus, dacă trimiterea datelor e realizată cu fetch() în loc de formular.

Bonus 11 (0.4p) Pentru fiecare produs, cand se da click pe containerul său să apară pe pagina de produse un modal box cu datele produsului (în loc sa folosim pagina separata dedicata produsului, sa apară direct pe pagina), modal-ul sa aiba un buton de iesire din el sau sa se inchida cand apasam in afara lui. Stilizarea modal-ului se va face cu culori din schema cromatica a site-ului.

Bonus 12 (Total: 1.2p) Se va implementa un sistem de oferte astfel:

- (0.5) Se consideră un interval de timp T la care se generează o nouă ofertă (care înlocuiește oferta anterioară). De exemplu, intervalul poate fi o zi (sau, pentru prezentarea bonusului, 1-2 minute). Va exista un fișier json, numit *oferte.json*, care va avea un obiect principal cu proprietatea "oferte", având o valoare de tip vector de obiecte. Obiectele din vector vor avea proprietățile "categorie", "data-incepere", "data-finalizare" și "reducere". La fiecare interval de timp T se va alege aleator o categorie (valorile posibile pentru categorie se vor prelua prin program din baza de date) și o valoare posibilă de reducere dintre valorile: 5,10,15,20,25,30,35,40,45,50. În JSON la începutul vectorului se va introduce noua ofertă, setând și data de începere și de finalizare (va începe din momentul generării și se va finaliza după intervalul T de timp). Nu se vor genera două oferte consecutive pentru aceeași categorie.
- (0.1) Pe prima pagina se va afișa oferta preluată din fișierul JSON (ar trebui să fie mereu prima din vector) sub formă de anunț pentru utilizatori (alegeți voi modul de afișare).
- (0.3) Oferta va avea și un temporizator care va afișa câte ore, minute și secunde mai sunt până expiră (temporizatorul se va actualiza la fiecare secundă). Ultimele 10 secunde vor fi marcate în mod diferit (de exemplu temporizatorul își schimbă culoarea sau generează un efect sonor). Când oferta expiră, aceasta dispăre și e înlocuită de noua ofertă generată.
- (0.2) De asemenea, în pagina de produse, se va afișa pentru produsele din categoria cu ofertă prețul vechi tăiat și alături de el, prețul nou redus cu procentajul din JSON.
- (0.1) Se va considera un interval de timp T2, după care nu ne mai interesează ofertele vechi, prin urmare, ofertele care au expirat în urmă cu mai multe minute decât T2 vor fi șterse din JSON.

Bonus 13 (0.15p) Se consideră un interval de timp T după care fișierele din folderul de backup nu mai sunt de interes, prin urmare vor fi șterse. În mod repetitiv, cu ajutorul unui apel setInterval se va verifica dacă există în folderul backup fișiere mai vechi decât numărul de minute din T, caz în care vor fi șterse.

Bonus 14 (0.3p) Cel mai ieftin produs. În pagina de produse se va marca cel mai ieftin produs din **fiecare** categorie. Modul de marcare îl alegeți voi însă trebuie să fie clar pentru utilizator (de exemplu să scrieți că este cel mai ieftin și să marcați cumva acel mesaj ca să atragă atenția)

Bonus 15 (0.05) În pagina cu produsele se va scrie și **numărul total de produse afișate** (acest număr se va modifica în urma filtrărilor).

Bonus 16 (0.3) Produse similare. Pe pagina descriptivă a unui produs unic se va crea o secțiune de produse similare. Produsele vor fi selectate prin program din baza de date după un criteriu ales de voi (de exemplu au o specificație comună sau sunt din aceeași categorie).

Produsele similare vor avea un afișaj sumar (nume, imagine, eventual câteva caracteristici mai importante). La click pe numele sau imaginea unui produs similar, utilizatorul va fi dus pe pagina celui produs.

Bonus 17 (total 1p) Seturi de produse. Se va realiza astfel:

- a. (0.2) Se va crea un tabel numit "seturi" care va avea coloanele: id, nume set, descriere set. Se va mai crea un tabel numit asociere_set care va avea coloanele id, id_set, id_produc. Tabelele se vor popula astfel încât să existe minim 5 seturi de produse. Un set trebuie să cuprindă minim două produse.
- b. (0.3) În meniul de produse va exista și opțiunea seturi, care va duce spre o pagină specială unde sunt afișate aceste seturi. Modul de afișare va fi ales de voi, dar utilizatorul va avea posibilitatea de a accesa pagina proprie a oricărui produs din set, făcând click pe nume/imagine.
- c. (0.2) Afișarea unui set va cuprinde și prețul acestuia care e calculat prin program astfel: se face suma prețurilor produselor din set și se aplică o reducere de $\min(5, n) \cdot 5\%$, unde n e numărul de produse din set.
- d. (0.3) În pagina proprie a fiecărui produs se vor afișa toate seturile din care acesta face parte, împreună cu prețurile lor. Utilizatorul va avea posibilitatea de a accesa pagina proprie a oricărui produs din set, făcând click pe nume/imagine.

Bonus 18. (0.2) Produse noi. Se va considera un interval T de timp (de exemplu $T = 1$ an - dar pentru prezentare îl veți alege mai scurt) în care un produs e considerat nou pe site; deci produsul e nou, dacă diferența între momentul curent și momentul adăugării este mai mică sau egală cu T . Produsele noi vor avea un marcaj (de exemplu, cuvântul "NOU", stilizat să atragă atenția, pus în containerul lor, pe pagina de produse). De asemenea, într-o secțiune specială, pe prima pagină se vor lista în ordine invers cronologică adăugării cele mai noi produse (de exemplu, pentru a le face reclamă). Produsele vor fi listate cu un link asociat, astfel încât utilizatorul să poată accesa direct pagina produsului. Marcajul de produs nou va fi afișat și pe pagina proprie produsului.

Bonus 19. (0.2) Orarul să poată fi vizualizat pe orice pagină a site-ului (fără a schimba pagina). De exemplu la click pe butonul/linkul de orar să apară un container pe pagină care apoi poate fi închis, dar poate și să dispară singur după un timp, în care este afișat tabelul cu orarul de funcționare. Ziua curentă va fi marcată diferit în tabel (de exemplu cu o culoare de fundal diferită), de asemenea sub tabelul cu orarul se va afișa dacă acum firma e deschisă ori închisă în funcție de **data și ora curentă**. Pentru orar, ca să fie vizualizat ușor pe orice pagină se va face un fragment ejs separat. Orarul va avea toate zilele săptămânii scrise, și intervale orare pentru fiecare, astfel încât macar la anumite ore din anumite zile, firma să fie închisă (studentul va modifica la prezentare tabelul cu orarul pentru a arăta cum se schimbă afișajul pentru "închis/deschis").

Bonus 20. (total= 1.5p; e scris în dreptul fiecărui subpunct câte puncte are alocat din total) Se va implementa posibilitatea de a compara două produse (puteți compara toate specificațiile sau alegeți un subset) astfel:

- (0.1p) Pe pagina fiecărui produs, dar și în tabelul de produse va exista un buton cu textul "compară".
- (0.2p) La click pe buton, pentru primul produs, se va crea un container cu id-ul "container-comparare" cu poziție fixă în care va fi afișat numele primului produs de comparat.
- (0.2p) Din momentul în care apare containerul "container-comparare", se va vedea pe orice pagină de produs (care descrie un produs anume) cât și pe pagina cu toate produsele. Containerul nu se va vedea pe paginile care nu afișează produse. Containerul va rămâne afișat în urma acțiunii de reîncărcare a paginii, dar și de închidere și reintrare pe pagină.
- (0.1) Totuși dacă timp de o zi nu s-a mai făcut click pe vreun buton de comparare, containerul va fi ascuns automat.
- (0.3) Va exista posibilitatea de a șterge oricare dintre produsele de comparat (de exemplu, cu un buton în dreptul numelui). Dacă erau 2 produse și s-a șters unul, butoanele cu textul "compară" redevin active. Dacă se șterg ambele produse dispare containerul cu id-ul "container-comparare".
- La click pe butonul "compară" când există deja un produs de comparat se vor întâmpla următoarele:
 - (0.2) Toate butoanele cu textul "compară" se dezactivează. Când se vine cu cursorul pe unul astfel de buton dezactivat apare în dreptul cursorului un mesaj informativ cu textul "ștergeți un produs din lista de comparare".
 - (0.1) Numele celui de-al doilea produs va fi afișat în containerul "container-comparare" alături de primul nume. Lângă cele două nume va fi un buton cu textul "afișează".
 - (0.3) La click pe butonul "afișează" se va deschide o nouă fereastră în care se afișează în paralel caracteristicile celor două produse (câte un rand de tabel pentru fiecare caracteristică și câte o coloană pentru fiecare produs).

Etapă 7 (bootstrap avansat, cookie-uri, pregătirea sistemului de utilizatori) (punctaj recomandat 2.4p)

25.05.2026

02.06.2026

(0.3p) Task bootstrap_js

Introduceți o componentă de tip acordeon (accordion - din Bootstrap) pentru descrierea produselor (și eventual alte detalii). Inițial acordeoanele sunt închise. Memorați în localStorage sau într-un cookie ce acordeon a fost deschis/inchis de către utilizator și la următoarea intrare pe pagina afișați acordeoanele așa cum le-a lăsat utilizatorul când a ieșit de pe pagina (de exemplu dacă le-a deschis, să se afișeze la refresh tot deschise).

(0.3p) Animatie-banner

Bannerul cu disclaimerul despre faptul că este un proiect școlar, va avea poziție fixă, aliniat la colțul din stanga jos al ecranului. Va avea o formă patratică cu latura egală cu un sfert din width-ul ecranului. Dimensiunea literelor va depinde și ea de dimensiunea lățimii ecranului.

Inițial nu se va vedea. La încărcarea paginii, se va scala de la 0, la dimensiunea stabilită, pornind din colțul din stânga jos. De asemenea va porni de la opacitate 0 pana la opacitate 0.75, schimbânduși treptat și culoarea de background. Animația va dura 5 secunde. După animație bannerul va rămâne pe ecran așa cum a fost setat la keyframe-ul 100%. Culoarele bannerului le alegeți conform schemei cromatice alese pentru site.

În banner va mai fi și textul "Acceptați cookie-uri?" (sau ceva asemănător) cu butonul "ok" (tot în banner). La click pe buton se setează un cookie care memorează faptul că a fost acceptat bannerul și bannerul va dispărea și nu se va mai afișa până expiră cookie-ul (afisarea după expirare se va face doar în caz de reload, nu apare automat într-o pagină deja deschisă). Cookie-ul va expira după o săptămână (deși la prezentare veți seta mai puțin, gen 5-6 secunde ca să îl putem verifica). Veți avea și o funcție deleteCookie(nume) care șterge cookie-ul cu acel nume, ca să putem verifica ușor funcționalitatea, respectiv o funcție deleteAllCookies() care șterge toate cookieurile setate.

Mai creați un cookie pe site, de exemplu: ultimul produs accesat, ultimele filtre setate (pe care le setați automat când intră utilizatorul pe pagina de produse), ultima pagină accesată, data ultimei cumpărături efectuate). Afișați ceva referitor la acest cookie undeva pe site, de exemplu în pagina de profil a utilizatorului sau pe prima pagină.

Alți parametri ai bannerului precum dimensiunea sau tipul de font, padding-ul culoarea de text etc, sunt lăsați la alegerea voastră.

Bannerul va fi static (fără animație) pe ecran mediu și mic.

- (obiectivele din cerința: animație css+cookies). Pentru banner veți folosi un element de forma :

```
<p id="banner">Acesta este un proiect scolar. <span id="mesaj-cookies">Acceptati cookie-urile de pe site? <button id="ok_cookies">Ok</button></span></p>
```

Cadru pentru sistemul de utilizatori:

(0.5p) Clasa AccesBD (se rezolvă la curs/laborator - punctele se dau de fapt la prezentare pe explicarea funcțiilor implementate împreună):

Se va implementa într-un fișier javascript dedicat o clasă pentru accesarea bazei de date; să o numim, de exemplu, AccesBD. Această clasă va urma **design pattern-ul Singleton** și va avea următoarele metode și proprietăți:

- o proprietate statică numită instanța, care va conține unica instanță a clasei. Inițial are valoarea null.
- un constructor care va arunca o eroare dacă deja a fost instanțiată clasa.
- una sau mai multe metode de inițializare a bazei de date prin care se oferă datele de autentificare (utilizator, parola, baza de date, port). Aceste metode vor salva în proprietatea client obiectul corespunzător conexiunii prin care se realizează cererile către baza de date. Se va scrie și un getter pentru client.
- o metodă getInstanța() care creează o instanță, dacă nu a fost deja creată, și o atribuie variabilei statice instanța. În această metodă se va inițializa și conexiunea la baza de date. Metoda va returna o referință către instanță.
- metodele de accesare și modificare a datelor. Fiecare dintre ele va primi ca parametru un obiect cu proprietățile descrise mai jos (care vor fi folosite pentru crearea query-ului corespunzător), dar și o funcție callback cu 2 parametri (err - setat dacă există o eroare, rez- setat dacă există un rezultat) prin care se va procesa rezultatul query-ului
 - select(obiect, callback). Parametrul obiect va avea ca proprietăți numele tabelului, un vector cu câmpurile selectate, un vector cu condițiile de selectare (de exemplu un vector de forma ["pret>50", "nume like 'a%'"]).
 - selectAsync(obiect). O metodă asincronă de selectare a datelor din tabel. Parametrul obiect va avea ca proprietăți numele tabelului, un vector cu câmpurile selectate, un vector cu condițiile de selectare (de exemplu un vector de forma ["pret>50", "nume like 'a%'"]). Metoda va returna rezultatul query-ului.
 - update(obiect, callback). Parametrul obiect va avea ca proprietăți numele tabelului, un vector cu câmpurile de setat și un vector cu valorile corespunzând câmpurilor (de lungime egală cu cea a vectorului de câmpuri), un vector cu condițiile de selectare a rândurilor care vor fi actualizate.
 - insert(obiect, callback). Va insera o înregistrare în tabel. Parametrul obiect va avea ca proprietăți numele tabelului, un vector cu câmpurile tabelului și un vector cu valorile corespunzătoare acelor câmpuri (alternativ se poate da ca parametru un obiect în care proprietățile sunt câmpurile și valorile proprietăților să fie valorile de inserat).
 - delete(obiect, callback). Parametrul obiect va avea ca proprietăți numele tabelului, un vector cu condițiile de selectare a rândurilor care vor fi șterse.

Fișierul javascript va exporta această clasă.

(0.05p) Drepturi (se rezolvă parțial la curs/laborator - punctele se dau de fapt la prezentare pe explicarea funcțiilor implementate împreună):

Realizați un fișier numit drepturi.js în care definiți un obiect (numit **Drepturi**) cu drepturile posibile. Numele proprietăților vor fi identificatori pentru drepturi, iar valorile lor vor fi obiecte de tip Symbol. Trebuie să aveți în fișier minim 7 drepturi (**trebuie să adăugați voi restul**). Aceste drepturi se referă la vizualizare, modificare, ștergere sau adăugare de date. Veți lega aceste drepturi de restul de cerințe.

Fișierul javascript va exporta obiectul **Drepturi**.

(0.25p) Clasa Roluri (se rezolvă la curs/laborator - punctele se dau de fapt la prezentare pe explicarea funcțiilor implementate)

împreună):

Realizați un fișier numit `roluri.js` în care definiți o clasă de bază `Rol`, cu subclasele `RolClient` (pentru clientul logat), `RolAdmin` (pentru administratorul site-ului care are absolut toate drepturile), `RolModerator` (pentru moderatorul site-ului care are toate drepturile legat de utilizatori dar nu poate cumpăra de pe site și nici nu poate vizualiza/modifica/adăuga/șterge produse). Clasa `Rol` va avea proprietatea publică `cod` în care se va găsi stringul corespunzător rolului, așa cum apare în tabel (de exemplu "admin" pentru rolul de administrator, sau "comun" pentru rolul de client).

Toate clasele de tip `Rol` vor avea un getter care returnează lista lor de drepturi (simboluri din clasa `Drepturi`, din `drepturi.js`).

Clasa `Rol` are metoda `areDreptul(drept)` care verifică în lista de drepturi posibile, dacă există dreptul dat ca parametru și returnează `true` sau `false`.

(design pattern:factory). Se va implementa o clasă `RolFactory` cu metoda statică `creazaRol(tip)` care primește codul unui `Rol` și returnează obiectul `Rol` corespunzător.

Fișierul javascript va exporta clasele `Rol` și `RolFactory`.

(0.7p) Clasa Utilizator (se rezolva la curs/laborator - punctele se dau de fapt la prezentare pe explicarea funcțiilor implementate împreună):

Se va implementa într-un fișier javascript dedicat clasa `Utilizator`. Pentru această clasă se vor defini:

- proprietăți corespunzătoare câmpurilor din tabelul utilizatori (fiecare coloană va avea o proprietate asociată).
- Se va defini un constructor care setează valori pentru aceste proprietăți prin intermediul unui obiect dat ca parametru (ca în exemplul de la curs). Proprietățile obiectului parametru vor corespunde proprietăților clasei și vor fi folosite pentru inițializarea acestora. Constructorul trebuie să poată fi apelat și fără parametri, caz în care unele proprietăți vor avea valori implicite (default).
- Se vor implementa metode de verificare a datelor. De exemplu: o metoda `verificaNume()` care verifică dacă numele utilizatorului respectă formatul cerut (menționat în altă cerință), o metoda `verificaUsername()` care verifică stringul care ar trebui să fie username-ul și eventuale alte funcții de verificare necesare pentru validările cerute la alte subpuncte.
- Metode de interacționare cu baza de date:
 1. O metodă `modifica()` care primește un obiect cu noile date ale utilizatorului și modifică înregistrarea din tabel corespunzătoare lui. Aruncă o eroare dacă utilizatorul nu există.
 2. O metoda `salvareUtilizator()` care înregistrează utilizatorul. Dacă username-ul mai există, va arunca o eroare cu un text explicativ.
 3. O metodă `sterge()` care șterge din tabel utilizatorul curent și aruncă o eroare dacă acesta nu există.
 4. O metodă statică sincronă, `getUtilizDupaUsername`, care caută un utilizator după username (dat ca parametru). Aceasta va mai primi și un obiect cu niște câmpuri setate și o funcție callback care va folosi acel obiect. De exemplu obiectul poate conține parola introdusă de utilizator la login, iar funcția callback verifică dacă parola corespunde username-ului dat.
 5. O metodă statică asincronă, `getUtilizDupaUsernameAsync(username)` care returnează obiectul de tip utilizator corespunzător username-ului dat ca parametru. Returnează `null` dacă nu găsește un astfel de utilizator în tabel.
 6. O metodă statică sincronă `cauta(obParam,callback)` care primește ca prim argument un obiect ale cărui proprietăți au aceleași nume cu cele ale instanțelor clasei `Utilizator`. Unele proprietăți din argumentul obiect pot lipsi (au valoare nedefinită), caz în care se va căuta doar după proprietățile inițializate (de exemplu, dacă obiectul e `{username: "ionel"}` va căuta doar după username. Metoda primește și un callback care are ca argumente: `err` (care va conține un mesaj de eroare dacă a dat eroare `query-ul`) și `listaUtiliz` (un vector cu obiecte de tip utilizator; poate fi și `vid` dacă nu s-a găsit niciun utilizator cu caracteristicile cerute). Callbackul poate fi folosit pentru procesarea utilizatorilor găsiți.
 7. O metodă asincronă `cautaAsync(obParam)` care primește ca argument un obiect ale cărui proprietăți au aceleași nume cu cele ale instanțelor clasei `Utilizator` (ca la subpunctul anterior). Metoda returnează o listă de obiecte de tip utilizator corespunzătoare caracteristicilor din `obParam`. Lista poate fi și `vidă`, dacă nu s-au găsit utilizatori care să corespundă condițiilor.
 8. O metodă `areDreptul(drept)` care primește o valoare din `drepturi.js` și, în funcție de rolul utilizatorului, returnează `true`, dacă are acel drept și `false` dacă nu.
 9. O metodă `trimiteMail(subiect, mesajText, mesajHtml, atasamente=[])` care trimite un e-mail utilizatorului, cu subiectul și textul dat (atât ca *plain text* cât și ca HTML și o listă de atașamente).

Fișierul javascript va exporta clasa `Utilizator`.

(0.3p) Veți adăuga **JSDoc** (în care se explică scopul funcției, tipul parametrilor și ce returnează) pentru toate clasele și funcțiile din modulele proprii. Se vor adăuga și pentru funcțiile (care nu sunt callback) definite în programul principal. Se acceptă și în TypeScript.

Bonusuri:

Bonus 1. (0.3p) Adăugați în funcțiile `select`, `update` și `delete`, posibilitatea de a avea operatorul logic "or" între condiții. De exemplu, să putem avea și un query de forma "select campuri from tabel where camp1=val1 or camp2=val2 and camp3=val3". Indicație: se poate face cu un

vector de vectori de condiții [["a=10", "b=20"], ["c=30", "d=40", "e=50"]] va avea aplicat "and" pe vectorii interni și "or" pe elementele vectorului mare rezultând într-o condiție where de forma "where a=10 and b=20 or c=30 and d=40 and e=50"

Bonus 2. (0.4p) Folositi un ORM precum Sequelize (<https://sequelize.org/>) in locul query builder-ului scris impreuna la laborator. **Atentie! Pentru ca acest bonus sa fie punctat este obligatoriu sa aveti un pattern de tip singleton implementat in alta parte in cadrul proiectului!**

Bonus 3. (0.4p) Filtre persistente. Se va afișa în pagina de produse și un checkbox cu eticheta "salveaza filtrare". Dacă acesta se bifează, ultima filtrare făcută cu checkboxul bifat va fi salvată în localStorage (sau cookie) și la reîntrarea pe pagină filtrele vor setate pe valorile din ultima filtrare și se vor afișa produsele corespunzătoare lor. La click pe butonul de resetare a filtrării, se debifează și checkboxul de salvare a filtrării și se șterge din localStorage sau din cookie filtrarea anterioară. Indicație: salvăm în localStorage/cookie valorile filtrelor. La încărcarea paginii setăm în inputuri acele valori și reapelăm funcția de filtrare, astfel încât să vedem rezultatele filtrării anterioare

Etapa 8 Inregistrare Utilizatori, Login, Logout

02.06.2026

Task utilizatori

Realizați următoarele cerințe referitoare la administrarea utilizatorilor

- Să se creeze un tabel de utilizatori cu următoarele informații: id(cheie unica, cu auto-increment), username, nume, prenume, e-mail, parolă, data nașterii, data înregistrării (cu valoarea default egală cu momentul curent), culoare_chat (default valoarea "black"), rol (cu valoarea default "comun"), telefon, imagine_profil .
- Va exista un formular cuprinzând câmpurile: nume (required), prenume(required), username(required), parolă(required), reintroducere parolă(required), e-mail(required), data nașterii (input de tip "date"), culoare_chat (input de tip "color" sau tagul "select") telefon, fotografie (nu e required, input de tip fișier)
- La trimiterea (submit) a formularului se vor face următoarele verificări:
 - Câmpurile required sunt completate. Verificarea se va face atât la server cât și la client.
 - Valorile din inputurile parolă și reintroducere parolă sunt egale (verificare doar la client).
 - (cu ajutorul RegEx): telefonul să poată să aibă sau nu simbolul "+" la începutul valorii, în rest să cuprindă numai cifre dintre care prima să fie 0, să fie un minim de 10 cifre.
 - alte 2 validări decise de voi (le veți enunța la prezentare)
 - La server se va verifica dacă userul deja există, caz în care nu se poate face înregistrarea.
- Dacă verificările nu sunt valide, se afișează un mesaj de eroare și userul nu e înregistrat. Mesajul de eroare va conține motivele clar enunțate pentru care a fost respinsă înregistrarea.
- Dacă datele din formular sunt corecte, se inserează datele userului în tabel. Parola se va insera criptată. Imaginea se va uploada la o cale stabilă de voi, iar în tabel se va memora doar calea imaginii. Userul va primi un mail de confirmare cu subiectul "Salut, stimat [nume]" și textul "Username-ul tău este [username] pe site-ul [nume site].", înlocuind câmpurile dintre paranteze drepte cu informația cerută. Numele site-ului va fi scris cu bold, italic și subliniat în același timp
- Mailul trimis la înregistrare, conține și un link de confirmare (cu un token generat special pentru acel utilizator). Utilizatorul nu se poate loga până nu confirmă pe mail (când încearcă să se logheze trebuie să primească un mesaj în care i se explică faptul că nu a confirmat mailul încă). Dacă utilizatorul nu accesează exact linkul dat pe mail (de exemplu dacă ar încerca un token falsificat) nu se consideră confirmat. **Formatul linkului de confirmare este:** [numeDomeniu]/cod_mail/[token1]-[token2]/[username]. Șirurile între paranteze drepte se vor înlocui. Șirul [numeDomeniu] e înlocuit cu numele de domeniu al site-ului, iar șirul [username] cu username-ul utilizatorului cu toate literele transformate în litere mari. Șirul [token1] va fi format din 50 litere mici aleatoare. Iar șirul token2 va fi un timestamp (exprimat în secunde de la 1 ianuarie 1970) al datei adăugării userului în tabel
- În header va exista un formular de login (deci se va afișa pe orice pagină) care cere un username și o parolă. Acesta se va afișa doar dacă nimeni nu este logat. Când utilizatorul este logat, formularul de login nu mai este afișat. În meniu apare opțiunea '\Logat: [username]\''. Când se vine cu cursorul pe el, apare ca un submeniu un link cu textul "delogare". La click pe el se realizează operația de logout, userul fiind redirecționat spre prima pagină și având în header înapoi formularul de login.
- De asemenea când utilizatorul e logat, site-ul îi afișează imaginea (uploadată la înregistrare) pe prima pagină.
- Va exista o pagină de profil a utilizatorului în care acesta poate să modifice datele despre el (cu excepția username-ului care nu mai poate fi schimbat, iar câmpul pentru username va fi readonly. Pagina va avea un formular cu datele deja completate (din tabel), cu excepția parolei. Pentru orice modificare, utilizatorul trebuie să scrie parola corectă. Implementați și posibilitatea de a-și schimba poza (se va suprascrive poza veche cu numele poza.png) dar și parola. În urma modificărilor, utilizatorul va primi un mail cu noile sale date (în cazul în care a schimbat adresa de mail, mesajul se va trimite pe adresa nouă).
- Minim doi dintre utilizatorii înregistrați vor primi rol de admin (modificați manual în baza de date). Utilizatorii cu rol de admin vor avea acces la o pagină specială, numită "Utilizatori", care apare în meniu doar dacă este un admin logat. În pagina respectivă se vor afișa toți utilizatorii într-un tabel (**cu excepția utilizatorului admin curent logat**). Tabelul cu utilizatorii trebuie stilizat folosind **bootstrap** (minim 3 clase de stilizare specifice tabelelor). Tabelul va conține o parte din datele lor (headerul tabelului va cuprinde numele coloanelor din tabelul din baza de date):
 - Username

- Nume (se va afișa numele de familie împreună cu inițiala prenumelui - nu tratăm cazul cu mai multe prenume)
 - Rol
 - Un buton cu textul "promovează/retrogradează" (la click se va promova utilizatorul de la rol comun la admin, sau, invers, de la admin la comun. Modificarea e făcută prin program în baza de date, iar utilizatorul va primi un e-mail cu mesajul: "Ai fost [promovat/retrogradat] la rolul [rol] ", înlocuind textul din parantezele drepte cu datele corespunzătoare. (Indicații: pentru fiecare utilizator se va face un formular cu un input hidden în care este scris id-ul lui, care este transmis către server pentru procesare.) **Se oferă bonus pentru cine rezolvă folosind AJAX/fetch.**
11. În meniul pentru admin va fi vizibil și un buton numit "administrare". La click pe el, adminul va intra într-o pagină în care va putea adăuga, modifica sau șterge produse. Alegeți voi modul de implementare al paginii.
 12. Se vor genera și include în site paginile "Termeni și condiții" și "Politică de confidențialitate".
 13. Se vor afișa utilizatorii online sub forma [prenume][nume] - [username] într-o listă ordonată, afișând utilizatorii în ordinea ultimei accesari (de la cel mai puțin recent la cel mai recent) scriși cu culoarea aleasă în formularul de înregistrare. Un utilizator se consideră online dacă a avut activitate în ultimele 8 minute. Pentru a memora accesările utilizatorilor se poate folosi un tabel în baza de date, conform celui din exemplu. Accesările mai vechi de o zi se vor șterge automat printr-un script care se repetă la fiecare oră.
 14. În meniul utilizatorului se va implementa și opțiunea de ștergere cont. La cererea de ștergere a contului, utilizatorul trebuie să introducă parola. Contul se va șterge doar dacă parola e corectă. După ștergerea contului utilizatorul va primi un mail de "la revedere".

Punctajul pentru fiecare subpunct este: $1(0.05) + 2(0.05) + 3(0.15) + 4(0.1) + 5(0.1) + 6(0.1) + 7(0.15) + 8(0.2) + 9(0.15) + 10(0.3) + 11(0.05) + 12(0.15) + 13(0.15)$

Bonusuri:

1. **(0.15p) Criptarea parolelor sa se faca pentru fiecare utilizator cu un alt salt string** (parola de criptare). Salt-ul se va genera aleator si se va memora in tabel pentru fiecare utilizator. La login se preia salt-ul din baza de date si se cripteaza cu ajutorul lui parola primita din input. Se compara parola criptata astfel cu cea din tabel.
2. **(0.2p) Dacă utilizatorul alege tema (light/dark sau o alta) în timp ce este logat, tema se va memora pe server pentru el** (de exemplu în tabel) și de fiecare dată când se loghează se încarcă tema preferată (se schimbă automat la login, fără ca utilizatorul să dea click pe butonul de schimbare a temei).
3. **(0.15p) Conturile cu e-mail necomfirmat** să primească o **notificare pe mail** după un timp t1 în care sunt anunțați că li se va șterge contul automat peste t2 timp dacă nu confirmă mailul. După ce trece t1+t2 timp de la creare, conturile se șterg prin program.
4. **(0.2p) Implementați o metodă de recuperare/resetare a parolei** la cererea utilizatorului (care a uitat parola și nu se mai poate loga).
5. **(0.3p) În cazul în care la înregistrare un utilizator introduce un username care mai există deja în tabel, acesta va primi sugestii de username-uri posibile.** Alegeți voi cum să fie generate username-urile (de exemplu pot primi un număr la final). Username-urile sugerate trebuie să nu existe în tabel deja.
6. **(0.2p) Dacă cineva greșește parola de k ori într-un timp t (de exemplu, k=5 si t=10min) atunci login-ul să fie blocat pentru un timp T(de exemplu T=1ora) iar utilizatorul să primească un mail de avertizare.**
7. **(0.2p) Se va implementa opțiunea de "site în mentenanță".** Se va crea un fișier JSON cu opțiuni pentru server. Se va crea un middleware cu app.use() pentru orice cerere (care va apela next()) pentru a transmite mai departe cererea după ce i-a aplicat opțiunile). Dacă în JSON-ul de opțiuni, opțiunea de site în mentenanță este true, atunci orice cerere primește ca răspuns o pagina speciala care anunta acest lucru, altfel e forwardata cu next().
8. **(0.1p) Afișați pe prima pagină pentru utilizator cât timp a trecut de la ultima lui logare pe site**
9. **(0.2p) Faceți ca sesiunea utilizatorului să expire după M minute(de exemplu M=1 ca să nu dureze mult la prezentare).** Implementați opțiunea de a **rămâne logat pe site automat** (de exemplu în funcție de ip) - adică utilizatorul nu mai este delogat automat dacă a bifat în profil această opțiune.
10. **(0.15p) Dacă utilizatorul nu a mai intrat de multă vreme** (alegeți voi intervalul de timp) pe site sau nu a mai cumpărat de multă vreme ceva, **să primească un mail promoțional** care să-i amintească de site. Mailul trebuie să cuprindă minim o imagine.
11. **(0.55p) Să existe minim 4 roluri posibile pentru utilizatori** (de exemplu, admin, user comun(client) si moderator, manager_produce) iar rolurile sa aiba drepturi asociate conform cerintelor de mai jos (în dreptul fiecărui subpunct e trecut și punctajul asociat din toatalul bonusului):
 - a. (0.05p) Se va crea un tabel cu rolurile. Tabelul va avea următoarele câmpuri: id, nume (numele rolului), descriere (o descriere succintă referitoare la rol). Se va completa cu rolurile menționate mai jos.
 - b. (0.05p) Se va crea un tabel cu drepturile posibile care va avea următoarele câmpuri: id, nume, descriere. Se va completa cu drepturile menționate mai jos. Pe baza acestuia se poate genera dinamic obiectul din drepturi.js (dat ca exemplu și menționat în cerințele obligatorii)
 - c. (0.05p) Se va crea un tabel de legătură între roluri și drepturi (de exemplu, numit roluri_drepturi) cu câmpurile: id, id_rol(foreign key), id_drept(foreign key). Se va completa cu asocierile menționate mai jos.
 - d. (0.025p) Se va crea un tabel de legatură între roluri și utilizatori cu câmpurile: id, id_utilizator, id_rol, data_inceput,

data_expirare

- e. (0.1p) În clasa Utilizator se va crea metoda areDreptul(identificatorDrept sau numeDrept), care va verifica dacă utilizatorul are dreptul respectiv (de exemplu, dreptul de afisare a listei de utilizatori, sau dreptul de adaugare produse. Functia, intern va verifica rolurile asociate utilizatorului care sunt active (data curentă este între data_inceput și data_expirare). Pentru fiecare rol va verifica existența dreptului respectiv.
- f. (0.05p) Rolurile propuse mai sus au următoarele drepturi:
- Admin: toate drepturile (se poate face chiar un caz special pentru el în funcția areDreptul(), ca să nu se mai scrie în tabelul roluri_drepturi atât de multe rânduri.
 - manager_produce are toate drepturile pe produse: creare, stergere, adaugare, editare
 - moderator are toate drepturile asupra utilizatorilor: creare, stergere, adaugare, editare, iar pentru anumite cerințe individuale, blocare. De asemenea, poate șterge comentariile utilizatorilor (acest drept se va implementa la un bonus ulterior)
 - comun (sau client) are dreptul de a cumpara (ce e în coșul virtual), de a da note produselor (dacă se va implementa acel bonus), de a lăsa un comentariu pentru produse.
 - utilizatorul nelogat (pentru el nu trebuie implementat ceva, dar există ca entitate abstractă) - nu are rol asociat, dar nu are niciun drept care implica serverul (cu excepția vizualizării datelor din pagini și adaugarea produselor în coș)
- g. (0.025p) Pentru testare veți avea 2 utilizatori admin, un utilizator moderator, un utilizator manager de produse, un utilizator care e și manager de produse și moderator și mai mulți utilizatori clienți.
- h. (0.2p) Adminul va avea o pagină (accesibilă printr-un drept prin care poate alocă roluri) în care poate selecta utilizatori și să le atribuie roluri, să le șteargă sau să editeze datele între care e valabil rolul. Pentru a seta datele, va avea două inputuri de tip "date", dar și câte un checkbox în dreptul fiecărui input, cu eticheta "perioada nedeterminată". Pentru perioada nedeterminată se va considera valoarea NULL în câmpul-dată respectiv. La submit, se va verifica faptul că data de început e înaintea datei de final.

12. (0.35p) **Data ultimei modificări a site-ului** (trecută acum în footer) va fi setată prin program la valoarea max(data_ultim_produc, data_max_modif_ejs). Data data_ultim_produc este cea mai recentă dată de adăugare a unui produs în tabel. Data data_max_modif_ejs este cea mai recentă dată a modificării unui ejș din folderul views și subfolderele sale. Când un utilizator intră pe site, dacă data ultimei accesări (după ip) sau logări este mai mică decât data ultimei modificări a site-ului, se va afișa în pagină un mesaj de atenționare cu textul "Au survenit modificări pe site.". Mesajul va fi afișat pentru 3 secunde (și apoi va dispărea), cu poziție fixă pe ecran.

13. (0.7p) Produse favorite.

- a. (0.1p) Fiecare produs va avea un buton/icon de adăugare la lista de favorite. Acest buton va fi afișat doar pentru utilizatorii logați. La click pe buton, produsul se salvează ca favorit.
- b. (0.05p) Salvarea produselor favorite se va face cu ajutorul unui tabel numit "favorite" cu câmpurile: id, id_produc, id_utilizator, data_favorit (data adăugării produsului la lista de favorite).
- c. (0.05p) Utilizatorii logați vor avea în meniu (poate fi și o opțiune de submeniu) secțiunea "favorite", care va duce la pagina /favorite unde se vor lista toate produsele favorite.
- d. (0.1p) În pagina de produse (dar și cea proprie fiecărui produs), produsele favorite vor avea un marcat special pentru utilizatorul logat. De asemenea, pentru fiecare produs se va afișa de către câți alți utilizatori a fost adăugat pe lista de favorite (de exemplu se va afișa "acest produs e preferat de 5 alți utilizatori"). Numărul se va actualiza automat la fiecare k secunde printr-un apel fetch(), de exemplu k=5.
- e. (0.2p) Utilizatorul admin va avea o opțiune nouă în meniu "top favorite" unde va vedea lista cu primele N produse (de exemplu N=20) în ordine descrescătoare a numărului de adăugări la "favorite" (de exemplu, primul produs a fost de 20 de ori favorit, următorul de 14 ori etc). În dreptul fiecărui produs va fi un input de tip number și un buton cu textul "Reumplere stoc" prin care admin-ul poate actualiza stocul pentru acele produse cu numărul trecut în input (inputul nu permite decât numere naturale nenule).
- f. (0.2p) În cazul în care un produs din lista de favorite a unui utilizator ajunge cu stocul sub STOC_MIN=5, utilizatorul va primi un e-mail de atenționare referitor la faptul că produsul favorit este pe terminate (mailul va cuprinde imaginea produsului, numele, descrierea și prețul). În mail va fi un link prin care, la click, va adăuga produsul în coșul virtual, iar utilizatorul va fi dus pe pagina coșului (cu produsul deja adăugat).

Cosul virtual ---- se vor afisa in curand cerintele

Bonusuri:

1. **(0.3) Starea comenzii.** Pentru fiecare factură din baza de date vom avea si campul stare cu urmatoarele valori: "comanda efectuata", "platit", "dat curierului", "in tranzit", "colet predat". In pagina de facturi, care poate fi accesata de catre admin, in dreptul fiecarei comenzi va fi un select simplu (sau un grup de radiobuttons) cu valorile posibile pentru starea comenzii. Setarea stării se va face printr-un apel de tip fetch(). Utilizatorul poate să vizualizeze schimbarea stării comenzii în pagina lui de comenzi.
2. **(0.3) Grafic d3js** Realizati un alt grafic (relevant) cu ce vreti voi, dar datele sa fie preluate din baza de date (Mongo sau Postgres) fie din activitatea utilizatorului salvata in localStorage sau cookies, **folosind de data asta d3js (nu tot chart.js)** (<https://d3js.org/>). Graficul trebuie sa aiba minim o tranzitie si asociate un comportament la click (de exemplu selectarea unei parti din grafic) si un comportament la mouseover (de exemplu, aparitia unui tooltip).
Se va crea un drept special pentru acest grafic, care se va adauga doar utilizatorului comun.
3. **(0.4) Sistem reduceri programate.** Reducerile programate se vor aplica în situații speciale descrise mai jos. Acestea nu vor schimba prețul produsului în baza de date, ci doar afișajul. Cerințele de implementare sunt următoarele:
 - a. **Afisare.** Prețul vechi (real, cel din tabel) se va afișa tăiat prin inserarea lui în tagul <s>. Prețul nou, redus, va fi inserat cu ajutorul tagului <ins>. Prețul va fi calculat prin program prin aplicarea reducerii procentuale (de exemplu dacă prețul real e 100 și reducerea 10%, atunci prețul calculat e 90).
 - b. Se va crea o clasă numită ReducereProgramata care va avea propriul ei fișier js. Fișierul va fi un modul node care va exporta clasa.
 - c. De aniversarea înregistrării utilizatorului pe site sau de ziua lui de naștere, acesta va primi o reducere de 10% pentru toate produsele de pe site.
 - d. Va exista un json sau xml cu date importante în care se aplică automat reduceri. Aceste date trebuie să fie în număr de minim 4 (de exemplu: Crăciun, Black Friday, aniversarea firmei, prima zi de primavară etc.). La pornirea serverului, aceste date vor fi încărcate din json într-o proprietate statică a clasei
 - e. Când stocul unui produs este sub 5, se aplică o reducere automată de 25%.
 - f. Se pot aplica reduceri cumulate (se adună procentajele pentru fiecare tip de reducere).
4. **(0.6) Sistem vouchere**
Utilizatorii vor putea primi vouchere în urma unor evenimente descrise mai jos. Voucherele pot fi aplicate asupra coșului de cumpărături pentru reduceri. Cerințe de implementare:
 - a. Se va crea un tabel de vouchere cu următoarele câmpuri: id, id_utilizator(foreign key), valoare (numeric), mod_aplicare (tip creat de voi stil enum cu valorile posibile: "procentaj", "scadere", "gratis_produc_minim", "produse_identice"), și câmpul data_expirare. Valoarea unui voucher e considerată valoare fixă sau procentaj în funcție de tipul voucherului
 - b. Într-un fișier separat se va crea o clasa Voucher, care are drept proprietăți chiar câmpurile din tabel (id, nume, valoare, mod_aplicare) și metodele necesare pentru aplicarea voucherelor. Se pot folosi clase diferite pentru vouchere diferite aplicând un design pattern de tip factory.
 - c. Pe pagina coșului virtual se va afișa lista de vouchere (nefolosite încă și neexpirate) pentru utilizatorul logat. Fiecare voucher va avea un checkbox cu eticheta "aplică". La bifarea checkboxului voucherul este marcat (de exemplu își schimbă culoarea de background sau de text) iar la debifare marcajul dispare.
 - d. Ordinea și modul de aplicare al diverselor tipuri de voucher este următorul:
 - i. Voucherul se aplică pe prețul afișat din coș (după ce a fost eventual modificat - în cazul în care s-au implementat și alte cerințe(obligatorii sau bonus) care precizau modificarea prețului)
 - ii. Întâi se aplică voucherul de tip "gratis_produc_minim". Acest tip de voucher se poate aplica o singură dată per comandă și numai pentru comenzi de măcar 2 produse. Pentru produsul cu prețul cel mai mic se va scădea prețul din total
 - iii. Apoi se aplică voucherul de tip produse_identice, care scade un anumit procentaj din prețul produselor cumpărate în număr mai mare de 1.
 - iv. Se aplică apoi voucherul de tip procentaj, care scade un procentaj din întreaga sumă de plată
 - v. Ultimul tip de voucher aplicat este cel de tip "scadere", care scade o valoare fixă din suma de plată.
 - vi. Va exista un tabel de legătură între utilizatori și vouchere, care indică pentru fiecare utilizator ce voucher are, data la care l-a primit și data expirării, și statusul voucherului (folosit sau nu). Alegeți voi cum primesc utilizatorii vouchere, dar să fie într-un mod automat (de exemplu le oferă adminul, sau primesc un mail-promotie cu un lick care le oferă voucherul etc).
5. **(0.3) Anunță revenirea în stoc.** Pentru produsele cu stoc 0 se va afișa un buton (doar pentru utilizatorii logați) cu textul "anunta cand revine in stoc". In momentul în care stocul devine diferit de 0, utilizatorii care au făcut click pe acel buton pentru acel produs vor primi un mail în care sunt notificați că a reapărut produsul. Va exista un tabel de legătură care memorează perechi de forma utilizator-produc și data la care s-a facut cererea. După trimiterea mailului de notificare înregistrarea corespunzătoare perechii utilizator-produc dispare. Dacă produsul e cu totul ster din baza de date, toți utilizatorii care cereau notificarea revenirii în stoc vor primi un mail în care sunt anunțați că produsul e pierdut pe vecie (RIP).

Etapa 10 - aspect (pana la 1p)	